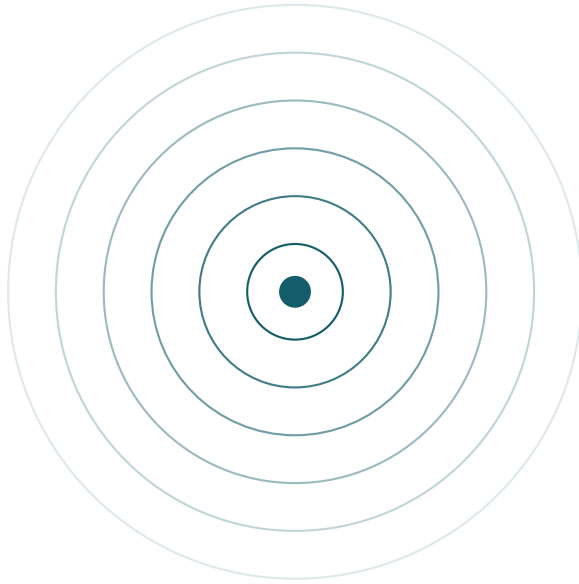


From your laptop to machines that serve everyone



Infrastructure Engineering from Zero

A Practical Guide

FIRST EDITION

Infrastructure Engineering from Zero

*A Practical Guide to Containers, Pipelines,
the Cloud, and Infrastructure as Code*

FIRST EDITION

Infrastructure Engineering from Zero: A Practical Guide

Copyright © 2026 The Author. All rights reserved.

Revision History for the First Edition

2026-09-26 First release

The programs in this book are included for their instructional value. They have been tested with Docker 29, act, Terraform 1.16, and Python 3.12 on macOS 26, but no warranty, express or implied, is offered with respect to them, and no liability is accepted for any damages arising from their use. Third-party services and tools described here change over time; check their current documentation.

Typeset in Crimson Pro, Source Sans 3, and Source Code Pro with XeLaTeX. Source: Markdown, converted with pandoc.

Contents

Preface	vii
Who This Book Is For	vii
How This Book Is Organized	vii
How to Use This Book	viii
What You Need Installed	viii
Conventions Used in This Book	ix
 Part I. Understand	 1
 1. From “Works on My Machine” to “Works for Everyone”	 2
Why “Works on My Machine” Happens	2
What Infrastructure Is	3
Environments	4
The DevOps Idea	4
The Cloud Idea	5
Lab: Two Machines, Two Answers	5
Summary, Key Terms, and Review Questions	7
 2. Containers: Shipping the Whole Kitchen	 8
Ship the Kitchen, Not the Recipe	8
Just Enough Operating System	8
Images and Containers	9
Layers	10
Containers Compared with Virtual Machines	10
Registries	11
Lab: Your First Containers	11
Summary, Key Terms, and Review Questions	14
 3. The Cloud: Renting Computers by the Minute	 15
Renting Instead of Buying	15
Three Ways to Rent: IaaS, PaaS, and Serverless	16
Regions and Zones	16
Compute: Where Code Runs	17

Storage: Where Data Lives	17
Networking: How Traffic Reaches It	18
Paying for What You Use	18
Lab: Pay-per-Use on Paper (and in Python)	19
Summary, Key Terms, and Review Questions	20
4. Pipelines: From a Commit to Production	22
The Nervous Afternoon	22
Just Enough Git	22
The Pipeline	23
Continuous Integration	23
Continuous Delivery and Continuous Deployment	23
Artifacts: Build Once, Deploy Many	24
Environments in a Pipeline	24
Deployment Strategies	24
Rollback	25
Lab: A Pipeline in Twenty Lines.	25
Summary, Key Terms, and Review Questions	27
Part II. Use	29
5. Docker in Practice	30
Meet tinyapp.	30
The Dockerfile.	31
Building and the Cache	32
Running, Ports, and Logs.	33
Volumes: Data That Survives	33
Configuration and Secrets	33
Smaller and Safer Images	34
Compose: Several Containers Together.	34
Lab: Build, Run, and Compose	35
Summary, Key Terms, and Review Questions	38
6. Your First Pipeline	39
GitHub Actions in One Picture	39
Running Workflows Locally with act	40
A Workflow That Runs the Tests	41
Building and Publishing the Image.	42
Secrets in CI.	43
Lab: Tests on Every Push	43
Summary, Key Terms, and Review Questions	45
7. Cloud Building Blocks.	46
The Few You Actually Need	46
Accounts and IAM.	46

Virtual Machines	47
Object Storage	48
Managed Databases	48
Firewalls	48
Serverless Functions	48
Cost Alerts	49
Lab: Least Privilege with Local Object Storage	49
Summary, Key Terms, and Review Questions	52
8. Infrastructure as Code	53
Clicks Don't Repeat	53
Declarative: Say What, Not How	54
Terraform	54
The Workflow: init, plan, apply, destroy	55
State	55
Lab: Your First Terraform	56
Summary, Key Terms, and Review Questions	58
Part III. Apply	60
9. Lab: Ship a Small Python App in a Container	61
What You'll Build	61
Step 1: Test the App Before You Package It	62
Step 2: Teach the Image to Report Its Health	62
Step 3: Build a Versioned Image	63
Step 4: Run It and Watch It Become Healthy	63
Step 5: See What Unhealthy Looks Like	64
Step 6: Push the Image to the Registry	65
Step 7: Run It "Somewhere Else"	65
When Something Goes Wrong	67
Summary, Key Terms, and Review Questions	67
10. Lab: Deploy on Every Push	68
The Plan	68
Step 1: The Deploy Script	69
Step 2: The Workflow	70
Step 3: The First Deploy	71
Step 4: A Good Change Goes Live	72
Step 5: A Bad Deploy, Caught and Rolled Back	73
Step 6: Roll Back by Hand	74
Step 7: Fix Forward	74
How act Reaches Your Docker	75
Summary, Key Terms, and Review Questions	75

11. Lab: Rebuild Everything from Code	76
What You'll Build	76
Step 1: Describe the System	77
Step 2: Build It	79
Step 3: Change Something	80
Step 4: The Server Dies	81
Step 5: Destroy and Recreate from Nothing	82
Step 6: Protect the Data from Yourself	83
Step 7: See Where the Password Went	83
Summary, Key Terms, and Review Questions	84

Part IV. What Next **85**

12. Security, Speed, Failure, and Cost	86
Security: What Can Go Wrong, and What Limits the Damage	86
Speed: What Makes This Slow	87
Failure: How It Breaks, and How to Diagnose It	88
Cost: What Does This Cost Me?	89
Summary, Key Terms, and Review Questions	89
13. Where to Go from Here	91
Kubernetes and What It's For	91
Platform Engineering	91
What to Learn Next, in This Order	92
What to Skip for Now	92
Cleaning Up the Labs	93
A Last Word	93

Appendices **94**

A. Docker Cheat Sheet	94
Images and Containers	94
Compose	95
Dockerfile Instructions	95
B. CI and Terraform Cheat Sheet	96
GitHub Actions and act	96
Terraform Commands	97
Terraform Language	97
C. Glossary	98
Index	100

Preface

It works on my machine but breaks on the server, and I don't even own a server.

Almost every developer says this at some point. The code is fine and the tests pass, but when it has to run somewhere else, for other people, everything that was quietly true on your laptop stops being true. This book answers one question:

How does my code get from my laptop to machines that serve everyone, the same way every time?

The answer has three parts. You package the program together with everything it needs (containers). You let a machine do the boring, error-prone steps for you on every change (pipelines). And you describe the machines themselves in files, so you can rebuild them whenever you like (infrastructure as code). The cloud is where all of this usually runs, so you will learn what it is and what it charges for.

Who This Book Is For

You know Python basics: variables, functions, `if`, loops, and how to run a `.py` file. You have never used Docker, never written a pipeline, and never rented a server. That is fine. Every idea starts from zero, and the few operating-system and networking ideas you need (processes, ports, `localhost`) are explained when they first appear.

How This Book Is Organized

- Part I, *Understand* (Chapters 1–4) builds the mental model: what infrastructure is, what containers are, what the cloud sells, and how a pipeline moves a change to production. There is very little code.
- Part II, *Use* (Chapters 5–8) makes you a confident user of the four tools: Docker, GitHub Actions, the cloud's basic building blocks, and Terraform.
- Part III, *Apply* (Chapters 9–11) is three longer labs. You ship a small app in a container, deploy it on every push (including a failed deploy and a rollback), and rebuild the whole setup from code.

- Part IV, *What Next* (Chapters 12–13) looks at security, speed, failure, and cost, and tells you what to learn next and what to skip.

How to Use This Book

Read Part I in order, then do every lab. Infrastructure is learned with your hands: the moment a pipeline goes red, or Terraform rebuilds something you deleted, the idea sticks.

Everything runs on your own computer: no cloud account, GitHub account, login, or credit card. Steps that only exist in a real cloud (a web console, cloud permissions, a billing alert) are described in words.

A tested copy of every lab file is in the book's lab files: download *Infrastructure Engineering from Zero Lab Files.zip* from this book's entry on the Towards Intelligence CS & Engineering on-ramp (robotbrainlab.github.io/cse/on-ramp.html) and unzip it. It unpacks to a folder called `labs/`: `labs/tinyapp/` holds the small web app used from Chapter 5 onward, and `labs/chNN/` holds each chapter's own files. Every lab starts in its own fresh folder under `~/infra-labs/`, so none depends on files left by another. Use the copies to check your work, not to skip the typing.

When you meet a term you don't know, look in Appendix C (Glossary).

What You Need Installed

- Docker Desktop (macOS or Windows) or Docker Engine (Linux). The book was tested with Docker 20. On Windows, use it with WSL 2.
- `act` 0.2.89 or newer, which runs GitHub Actions workflows locally (`brew install act` on macOS; see the `act` documentation for other systems).
- Terraform 1.16 or newer.
- Python 3.12 or newer, `git`, and `curl`.
- About 5 GB of free disk space. Most of it is Docker images; `act`'s runner image alone is over 2 GB.

Check them with:

```
docker version --format '{{.Server.Version}}'
act --version
terraform version
python3 --version
```

Conventions Used in This Book

- **Bold** marks a new term the first time it is explained. Every such term is also in Appendix C.
- Constant width marks commands, file names, code, and anything you would type exactly.
- Shell commands are shown without a prompt, so you can paste them as they are. Output follows in a separate block. A line containing only ... means output was trimmed.
- Container IDs, commit hashes, timings, and dates in your output will differ from the book's.
- Three kinds of callout appear in the text:



A suggestion that saves time or trouble.



Something to be aware of.



A mistake that can cost you data, money, or security. Read these twice.

- Chapters 1–8 end with a Lab, then a Summary, Key Terms, and Review Questions. Chapters 9–11 are labs from start to finish.

PART I

Understand



What to ignore for now: Kubernetes, service meshes, and comparing every cloud provider's products.

From “Works on My Machine” to “Works for Everyone”

This chapter explains why code that runs fine on your laptop can fail somewhere else, and names the ideas the rest of the book is built on: infrastructure, environments, DevOps, and the cloud. By the end you will have watched the same file work on one machine and crash on another.

The problem. It runs perfectly on my laptop and crashes on the server.

The question. What is different about the server, and how do teams make every machine behave the same way?

Why “Works on My Machine” Happens

Think of a recipe. It works perfectly in your kitchen. You send it to a friend, and their cake sinks. Nothing was wrong with the recipe. Their oven runs hotter, they had a different flour, and they skipped an ingredient you never wrote down because you always have it in the cupboard.

Code is a recipe, and the machine it runs on is the kitchen. When you run a Python program, it silently depends on many things that are not in your `.py` file:

Hidden ingredient	Example of what differs
The Python version	3.12 on your laptop, 3.9 on the server
Installed libraries	FastAPI 0.141 locally, an older one there, or none
The operating system	macOS or Windows locally, Linux on almost every server
Settings	A password or a URL you set once on your laptop and forgot
Files and data	A folder or a database that only exists on your machine
The network	Your laptop can reach a service that the server cannot

Every one of these is a way for “it works on my machine” to become “it crashes on the server”. The fix is never “be more careful”. People forget. The fix is to make the hidden ingredients visible, write them down, and let machines set them up the same way every time. That is what this book teaches.

What Infrastructure Is

A city runs on things nobody looks at: roads, water pipes, power lines. Without them, nothing works.

Infrastructure is the same thing for software: everything your code needs in order to run for other people, apart from the code itself. That includes the machines, their operating systems, the network between them, where data is stored, and the steps that put your code onto those machines.

A server is simply a computer whose job is to run programs for others, usually without a screen or keyboard, in a building full of other servers called a data centre. There is nothing magic about it. Your laptop could be a server if it were always on and reachable.

Infrastructure engineering is the work of making all of this reliable and repeatable: the same result every time, on any machine, without a person clicking through steps from memory.

Environments

An **environment** is one complete place where your app runs, with its own machines, settings, and data. Most teams have several:

Environment	Who uses it	What it's for
Development	You	Writing code, usually on your laptop
Test	Automated tests	Checking each change
Staging	The team	A dress rehearsal that looks like production
Production	Real users	The real thing

A theatre company rehearses on the real stage, in costume, before opening night. **Staging** is that dress rehearsal: it should look as much like production as possible, so that surprises happen there instead of in front of the audience.

The values that change from one environment to the next (a database address, a password, a feature switch) are called **configuration**. Configuration belongs outside your code, so the same code can run unchanged in every environment. You will pass configuration into containers in Chapter 5.

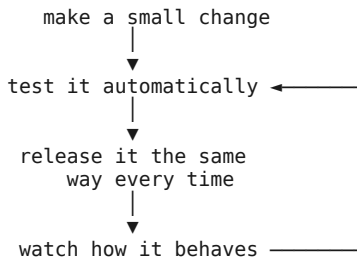


The more your environments differ, the more bugs hide in the differences. A good rule is “same code, same packaging, different configuration”.

The DevOps Idea

In many older companies, developers “threw code over the wall” to an operations team, who installed it on servers. Each side blamed the other when things broke, and painful releases happened a few times a year.

DevOps is the idea that building software and running it are one job, shared by one team, with as much of the work automated as possible. Its heart is the feedback loop: the time between making a change and learning whether it worked.



Short loops are safe loops. If you change ten lines and something breaks ten minutes later, you know where to look. If you change ten thousand lines and release them once a quarter, you don't.

This book gives you three tools that make the loop fast and safe:

1. *Package* the app with everything it needs, so it runs the same everywhere (containers, Chapters 2 and 5).
2. *Automate* the steps from a commit to a running app (pipelines, Chapters 4 and 6).
3. *Describe* the machines themselves in files, so you can rebuild them at will (infrastructure as code, Chapter 8).

The Cloud Idea

You don't need to own a server any more than you need to own a power station. The **cloud** means renting computers, storage, and ready-made services from a provider over the internet, and paying only for what you use, often by the second. Amazon Web Services (AWS), Microsoft Azure, and Google Cloud are the big three. Chapter 3 explains what they sell.

For now, one idea matters: in the cloud, machines are created by software in seconds, and they can disappear just as fast. So you can't rely on careful manual setup. You need everything to be repeatable.

Lab: Two Machines, Two Answers

You'll run one Python file on two "servers" and watch it work on one and crash on the other. The servers are containers, which Chapter 2 explains. For now, treat `docker run python:3.9-slim` as "borrow a clean Linux machine that has Python 3.9".

1. Make a folder for the book's labs and a file called `hello.py`:

```
mkdir -p ~/infra-labs/ch01 && cd ~/infra-labs/ch01
```

```
import platform

def describe(count):
    match count:
        case 0:
            return "no visitors yet"
        case 1:
            return "one visitor"
        case _:
            return f"{count} visitors"

print(describe(3), "on Python", platform.python_version(), platform.system())
```

2. Run it on your own machine:

```
python3 hello.py

3 visitors on Python 3.14.4 Darwin
```

Your version may differ; Darwin means macOS.

3. Run the same file on a machine with Python 3.9. The `-v` part lends your folder to that machine (Chapter 5 explains it). The first run downloads Python 3.9, which takes a minute.

```
docker run --rm -v "$PWD":/work -w /work python:3.9-slim python hello.py

File "/work/hello.py", line 5
    match count:
        ^
SyntaxError: invalid syntax
```

4. Run it once more with Python 3.12:

```
docker run --rm -v "$PWD":/work -w /work python:3.12-slim python hello.py

3 visitors on Python 3.12.14 Linux
```

Expected results: the same file works on your laptop and on Python 3.12, and fails with a `SyntaxError` on Python 3.9, because `match` only exists from Python 3.10. Notice also that the containers report `Linux`, not your laptop’s operating system. The code never changed. Only the kitchen did.

Summary, Key Terms, and Review Questions

Summary

- Code silently depends on things outside the file: the Python version, libraries, the operating system, settings, data, and the network.
- Infrastructure is everything your code needs to run for others. Infrastructure engineering makes it reliable and repeatable.
- Apps run in several environments. Staging should look like production; configuration is what differs between them.
- DevOps joins building and running software, and aims for short, safe, automated feedback loops.
- The cloud rents machines and services by use. Machines come and go, so setup must be repeatable.

Key Terms

Term	Meaning
Infrastructure	Everything code needs to run for others, apart from the code
Environment	One complete place where an app runs, with its own settings and data
Production	The environment real users use
Staging	A production-like rehearsal environment
Configuration	Settings that differ between environments, kept outside the code
DevOps	Building and running software as one automated job
Cloud	Computers and services rented over the internet, paid for by use

Review Questions

1. List four hidden ingredients a Python program depends on that are not in its .py file.
2. What is the difference between staging and production, and why should they be alike?
3. Why should configuration live outside the code?
4. What is a feedback loop, and why is a short one safer?
5. In the lab, why did the same file fail on one machine and work on another?

You understand this when you can look at a “works on my machine” bug and name which hidden ingredient differed between the two machines.

Containers: Shipping the Whole Kitchen

Containers are the most important idea in this book. This chapter explains what they are, how they relate to images, why images have layers, how containers compare with virtual machines, and where images are stored. Then you run your first containers.

The problem. The server has a different Python, different libraries, and a different operating system.

The question. Can I send my program together with everything it needs, so the server's differences stop mattering?

Ship the Kitchen, Not the Recipe

In Chapter 1, your recipe failed in a friend's kitchen. You could send a longer recipe with every detail. Or you could do something bolder: send the whole kitchen. A food truck arrives with its own oven, its own flour, and its own spices. It parks anywhere there is a road and electricity, and it cooks the same dish every time.

A **container** is a food truck for software. It is your program running together with its own private copy of everything it needs: the right Python, the right libraries, and the Linux files they expect. The server only has to provide the road and the electricity, which means a machine that can run containers.

Just Enough Operating System

To see what a container really is, you need three words.

The operating system (OS) is the software that manages a computer for all the programs on it: Linux, macOS, Windows. At its centre is the kernel, the part that talks to the hardware.

The kernel decides which program uses the processor, hands out memory, reads and writes files, and sends network traffic.

A process is one running program. Every process gets a number from the kernel, its process ID (PID). When you run `python3 hello.py`, the kernel starts a process, gives it a PID, and cleans up when it ends.

A container is not a small computer. It is an ordinary process that the Linux kernel fences off. Inside the fence, the process sees its own files, its own list of processes (in which it is often PID 1, the first and only one), and its own network. Outside the fence, it is just one more process on the machine.



The fencing is a Linux kernel feature, so containers are Linux programs. On macOS and Windows, Docker Desktop quietly runs a small Linux virtual machine and runs your containers inside it. That is why the containers in Chapter 1 reported `Linux`.

Images and Containers

A container starts from an **image**: a read-only package holding all the files the program needs, plus a note saying which command to run at start-up.

If you know Python classes, this picture helps: an image is like a class, and a container is like an object made from it. You write the class once. You can create many objects from it, each with its own state, and deleting an object doesn't touch the class.

	Image	Container
What it is	A package of files and a start command	A running (or stopped) process made from an image
Changes?	Never; you build a new one instead	Yes, it can write files while it runs
How many	One per version of your app	As many as you start
Lifetime	Kept until you delete it	Often seconds to days, then replaced

Containers are disposable. Anything a container writes into its own files disappears when the container is removed. That sounds alarming, but it's a feature: every container starts from the same clean image, so there is no slow build-up of manual changes. Data that must survive goes somewhere else, which Chapter 5 covers.

Docker is the most popular tool for building images and running containers. The `docker` command you type is a client. It sends your requests to the Docker daemon, a background service that does the real work of pulling images and starting containers.

Layers

An image is built in steps: start from a small Linux, add Python, install your libraries, copy in your code. Each step saves its changes to the files as a **layer**. The image is the stack of layers, like transparent sheets laid on top of each other: look down through them and you see one complete set of files.

writable layer (this container)		◀ gone when removed] read-only image layers, shared
your code: main.py	12 kB	
pip install: libraries	54 MB	
Python 3.12	45 MB	
Debian Linux files	110 MB	

Layers matter for two practical reasons:

1. *Sharing*. Ten images built on the same Python base share its layers. Your disk stores them once, and a download skips layers you already have.
2. *Speed*. When you rebuild an image, Docker reuses every layer whose inputs didn't change. Change one line of code, and only the top layer is rebuilt. Chapter 5 shows how to order the steps so this works in your favour.

A running container adds one thin writable layer on top. That is where its own changes go, and why they vanish with it.

Containers Compared with Virtual Machines

Before containers, the usual way to isolate programs was a **virtual machine** (VM): software that pretends to be a whole computer, with its own virtual processor, memory, and disk, running its own complete operating system, kernel included. A program called a hypervisor runs many VMs on one physical machine.

Picture houses and flats. Each VM is a separate house with its own foundations, plumbing, and wiring. Containers are flats in one building: each has its own locked door and furniture, but they share the foundations and the plumbing, which is the host's kernel.

	Virtual machine	Container
Contains	A whole OS, including a kernel	An app and its files; uses the host's kernel
Typical size	Gigabytes	Tens to hundreds of megabytes
Start time	Tens of seconds to minutes	Usually under a second
Isolation	Very strong	Good, but all share one kernel
Can run	Any OS	Linux programs on a Linux kernel

They are not rivals: in the cloud, your containers usually run inside VMs, which keep customers apart.

Registries

You build an image on your laptop. How does the server get it? Through a **registry**: a server that stores images and hands them out, like a library that lends out books. You push an image to a registry to upload it, and pull it to download it. Docker Hub is the default public registry; cloud providers each run their own, and in Chapter 6 you will run one on your own machine.

An image's full name tells Docker where to find it:

localhost:5001/tinyapp:1.0.0

registry repository tag

A **tag** is a human-friendly label for one version, such as 1.0.0 or 3.12-slim. When the registry part is missing, Docker assumes Docker Hub, so `python:3.12-slim` really means `docker.io/library/python:3.12-slim`.

Tags can be moved to point at a different image later. For an exact, unchangeable reference, every image also has a **digest**: a long fingerprint like `sha256:f77ac9e4...` calculated from its contents. The same digest always means exactly the same bytes.



A tag like `latest` can point at a different image tomorrow. For anything that must be repeatable, use a specific version tag, and for the strictest cases, the digest.

Lab: Your First Containers

You'll run containers, look inside them, see their layers, and prove that they are disposable.

1. Download (pull) an image. If you did the Chapter 1 lab, it is already there:

```
docker pull python:3.12-slim
```

```
3.12-slim: Pulling from library/python
Digest: sha256:
    ↪ f77ac9e44ae96ef2c90b8053ea08c31f8be030f824196b0ae4db6d462c84e51f
Status: Image is up to date for python:3.12-slim
docker.io/library/python:3.12-slim
```

2. Run a one-off container. `--rm` removes it when it finishes:

```
docker run --rm python:3.12-slim python -c "print('hello from inside a
    ↪ container')"

hello from inside a container
```

3. Look at the operating system inside it:

```
docker run --rm python:3.12-slim cat /etc/os-release | head -3

PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
```

Your laptop runs something else, but the container carries its own Debian files.

4. Ask the process for its PID:

```
docker run --rm python:3.12-slim python -c "import os; print('my process id
    ↪ is', os.getpid())"

my process id is 1
```

Inside its fence, your program is the first and only process.

5. Look at the image's layers, newest first:

```
docker history --format 'table {{.CreatedBy}}\t{{.Size}}' python:3.12-slim
```



```

CREATED BY                                SIZE
CMD ["python3"]                           0B
RUN /bin/sh -c set -eux;  for src in idle3 p... 16.4kB
RUN /bin/sh -c set -eux;  savedAptMark="$(a... 44.6MB
ENV PYTHON_SHA256=5c8462af5790baf43a321a1559... 0B
ENV PYTHON_VERSION=3.12.14                0B
...
# debian.sh --arch 'arm64' out/ 'trixie' '@1... 110MB

```

The `--format` option picks just two columns so the table fits. The bottom row is the Debian base. The 44.6 MB step installed Python. Rows of 0B only record settings.

6. Prove that containers are disposable. Give a container a name, and have it write a file:

```

docker run --name scratchpad python:3.12-slim sh -c "echo 'I was here' >
↳ /note.txt && cat /note.txt"

```

```
I was here
```

Now look for the file from a new container made from the same image:

```

docker run --rm python:3.12-slim cat /note.txt

cat: /note.txt: No such file or directory

```

7. The first container stopped but still exists. List all containers, including stopped ones, then remove it:

```

docker ps -a --filter name=scratchpad --format 'table
↳ {{.Names}}\t{{.Image}}\t{{.Status}}'
docker rm scratchpad

NAMES          IMAGE          STATUS
scratchpad     python:3.12-slim  Exited (0) Less than a second ago
scratchpad

```

Expected results: every container reports Debian Linux and PID 1, `docker history` shows the base layer at the bottom, and no other container can see `scratchpad`'s file, because it lived only in that container's writable layer.

Summary, Key Terms, and Review Questions

Summary

- A container is a normal Linux process fenced off by the kernel, with its own files, process list, and network.
- An image is a read-only package of files plus a start command. A container is a running instance of it, like an object made from a class.
- Images are stacks of layers. Layers are shared between images and reused when rebuilding.
- Virtual machines carry a whole OS; containers share the host's kernel, so they are smaller and start faster. In the cloud, containers usually run inside VMs.
- Registries store images. You push and pull them by name and tag; a digest names one exact image.

Key Terms

Term	Meaning
Container	A program running with its own private files, isolated by the kernel
Image	A read-only package of files and a start command, used to create containers
Layer	One step's worth of file changes in an image
Virtual machine (VM)	Software that imitates a whole computer, with its own OS
Registry	A server that stores and serves images
Tag	A movable, human-friendly version label on an image
Digest	A fingerprint that names one exact image

Review Questions

1. In your own words, why does a container stop “works on my machine” problems?
2. Using the class and object picture, how do an image and a container differ?
3. Why do images use layers? Give two benefits.
4. Name two differences between a virtual machine and a container.
5. What does `docker.io/library/python:3.12-slim` tell Docker, part by part?

You understand this when you can explain why a file written in a container vanishes with it.

The Cloud: Renting Computers by the Minute

This chapter explains what “the cloud” actually sells: three ways to rent computing power, regions and zones, the building blocks every app uses (compute, storage, and networking), and how the bill is calculated. The lab is a small Python cost model, because in the cloud every design decision is also a pricing decision.

The problem. I don’t own a server, and buying one feels wrong.

The question. How do I get a computer on the internet without buying one, and what exactly am I paying for?

Renting Instead of Buying

You could buy a car to get to the airport twice a year, or take a taxi. The taxi costs more per kilometre, but you pay only when you ride and someone else fixes the engine. Owning servers is owning the car: you pay up front, you buy enough for your busiest day so the machine is mostly idle, and a disk failing at 3 a.m. is your problem.

The cloud is the taxi. You rent on-demand: ask for a machine and get it in about a minute, then give it back and stop paying. You can also grow and shrink as traffic changes, which is called elasticity. The price per hour is higher than owning, but you never pay for the idle years.

Renting also splits the work. Under the **shared responsibility model**, the provider secures the buildings, the hardware, and its own services. You secure what you put on top: your code, your data, your settings, and who gets access. Most cloud security incidents happen on the customer’s side of that line, usually through a setting left open.

Three Ways to Rent: IaaS, PaaS, and Serverless

Think of pizza: make it at home, rent an equipped kitchen and cook it yourself, have a bakery bake your recipe, or buy one slice only when you're hungry.

	You manage	The provider manages	Examples
Your own servers	Everything, including the building	Nothing	A server in your office
IaaS	OS, runtime, app, data	Hardware, network, buildings	AWS EC2, Google Compute Engine
PaaS	App and data	Also the OS, runtime, scaling	Render, Heroku, Google Cloud Run
Serverless	Just your functions	Also servers, scaling to zero	AWS Lambda, Azure Functions

Infrastructure as a Service (IaaS) rents you virtual machines, disks, and networks. You get a bare Linux machine and do everything else: updates, containers, security. It is the most flexible option and the most work.

Platform as a Service (PaaS) runs your code or container image for you: “run this image with 512 MB of memory and send it traffic”. The platform handles the machines, restarts crashes, and adds copies when traffic grows.

Serverless goes further: the provider runs your code only when a request or event arrives. The best-known form is function as a service (FaaS), where each request runs one function. When nothing happens, nothing runs, and you pay nothing. The catch is the cold start: if your function hasn't run for a while, the first request waits while it starts, from a fraction of a second to several seconds.



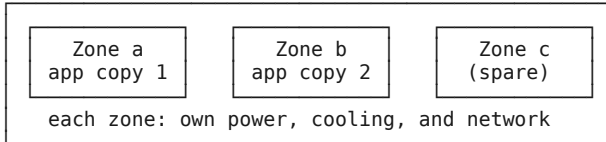
Containers fit all three. You can run them yourself on an IaaS machine, hand them to a PaaS, and several serverless services accept container images too. That is one reason this book starts with containers.

Regions and Zones

A **region** is a geographic area, such as Frankfurt or Mumbai, containing several data centres. Pick one close to your users, because distance adds delay, and one where the law allows your data to live.

Inside each region are several **availability zones**. A zone is one or more data centres with their own power, cooling, and network, a few kilometres from the others. A fire or power cut in one zone should not affect the others.

Region: eu-central (Frankfurt)



To survive a zone failing, run copies of your app in at least two zones. Surviving a whole region failing, which is rare, needs two regions and is well beyond this book.

Compute: Where Code Runs

Compute is the cloud’s word for processing power, in the three shapes you just met:

- An instance is a virtual machine you rent (IaaS). You choose an instance type, a fixed size such as “2 virtual CPUs and 4 GB of memory”, and a starting disk image with an operating system on it.
- A container service (PaaS) runs your images and adds or removes copies as needed.
- A function service (serverless) runs your code per request.

Storage: Where Data Lives

Apps store three kinds of thing, and the cloud has a different service for each:

Kind	What it is	Good for
Block storage	A virtual disk attached to one machine	The operating system, a database’s files
Object storage	Files (“objects”) stored by name in buckets, read and written over HTTP	Uploads, images, backups, logs
Managed database	A database the provider runs and backs up for you	Your app’s records

Object storage is the least like your laptop: no folders, no disk to mount. You send “store these bytes as photos/cat.jpg in bucket uploads” and later “give me photos/cat.jpg”. In return it is cheap, practically endless, and very durable. Amazon’s version, S3, is so common that other products copy its interface, which is how you’ll try it locally in Chapter 7.

Networking: How Traffic Reaches It

Just enough networking first. Every machine on a network has an IP address, such as 10.0.1.5, which works like a street address. One machine runs many programs, so each program that accepts connections listens on a numbered **port**, like a flat number in a building. A web app might listen on port 8000; a PostgreSQL database uses port 5432.

A cloud network is built from a few pieces:

- A **virtual private cloud** (VPC) is your own private network inside the provider, split into subnets. A public subnet can be reached from the internet; a private one cannot, which is where databases belong.
- A **firewall** decides which traffic may pass: for example, “allow port 443 from anyone, and allow port 5432 only from the app’s machines”. Many clouds call these rules security groups.
- A **load balancer** is the front door. It receives all incoming requests and spreads them across healthy copies of your app, skipping any that stop answering.

Picture a gated community: the VPC is the wall, the firewall is the guard with a list of who may enter, and the load balancer is the receptionist sending each visitor to a free desk.

Paying for What You Use

Every cloud service has a meter. The common units are:

Unit	Example
Time running	\$0.02 per hour for a small instance, billed by the second
Storage held	\$0.023 per GB per month in object storage
Requests	\$0.20 per million function calls
Data sent out	Egress, around \$0.09 per GB leaving the provider

(These are round example figures. Real prices vary by provider, region, and year.)

Two things surprise every beginner. The meter runs whether or not anyone uses the thing, so a forgotten test machine costs the same as a busy one. And small prices multiply: a tiny per-request price can become the biggest line on the bill at high traffic.



Cloud bills have no upper limit by default. Before you create anything in a real account, set a budget alert (Chapter 7) and learn how to delete what you create (Chapter 8).

Lab: Pay-per-Use on Paper (and in Python)

You'll compare two ways to pay for the same small app: one small instance running all month, or serverless functions billed per request. The prices are round example numbers, not any provider's real price list.

1. Create `~/infra-labs/ch03/cost.py`:

```
"""Compare two ways to pay for the same small app.

The prices are round example numbers, close to what big clouds
charge, but not any provider's real price list.
"""

HOURS_PER_MONTH = 730

# Option 1: a small virtual machine that runs all month.
VM_PRICE_PER_HOUR = 0.02

# Option 2: serverless, billed per request and per unit of work.
PRICE_PER_MILLION_REQUESTS = 0.20
PRICE_PER_GB_SECOND = 0.0000167
MEMORY_GB = 0.5          # memory given to each run
SECONDS_PER_REQUEST = 0.1

def vm_cost(requests):
    return VM_PRICE_PER_HOUR * HOURS_PER_MONTH # same price at any traffic

def serverless_cost(requests):
    per_request = requests / 1_000_000 * PRICE_PER_MILLION_REQUESTS
    work = requests * SECONDS_PER_REQUEST * MEMORY_GB * PRICE_PER_GB_SECOND
    return per_request + work

print(f"{'requests/month':>15} {'VM':>9} {'serverless':>11}")
for requests in [10_000, 1_000_000, 10_000_000, 50_000_000]:
    print(f"{'requests':>15,} {'vm_cost(requests):>9.2f} "
          f"{'serverless_cost(requests):>11.2f}"))
```

A GB-second is one gigabyte of memory held for one second, the usual way serverless work is measured.

2. Run it:

```
cd ~/infra-labs/ch03 && python3 cost.py
```

requests/month	VM	serverless
10,000	14.60	0.01
1,000,000	14.60	1.03
10,000,000	14.60	10.35
50,000,000	14.60	51.75

- Find the break-even point. Change the list of request counts to [14_000_000, 15_000_000] and run it again:

requests/month	VM	serverless
14,000,000	14.60	14.49
15,000,000	14.60	15.53

- Put the list back, change MEMORY_GB to 1.0, and run it once more. The 10-million row now shows 18.70 for serverless, above the instance's 14.60.

Expected results: serverless is almost free at low traffic and grows in a straight line with use; the instance costs the same at any traffic. With these prices they cross at about 14 million requests a month, and giving each function twice the memory makes serverless roughly twice as expensive per request. The lesson isn't which option wins. It's that you can, and should, work the numbers out before you build.

Summary, Key Terms, and Review Questions

Summary

- The cloud rents computing on demand. You pay more per hour than owning, but never for idle capacity, and you can grow and shrink quickly.
- The provider secures the platform; you secure what you put on it.
- IaaS rents machines, PaaS runs your app or container, and serverless runs your functions per request, with cold starts as the trade-off.
- Regions are places; availability zones are separate data centres within a region. Spread copies across zones to survive one failing.
- Apps use compute, storage (block, object, database), and networking (VPCs, subnets, firewalls, load balancers).
- Everything has a meter, including data sent out. Small prices multiply.

Key Terms

Term	Meaning
Shared responsibility model	The provider secures the platform; you secure what you put on it
Infrastructure as a Service (IaaS)	Renting virtual machines, disks, and networks
Platform as a Service (PaaS)	A service that runs your code or container for you
Serverless	Running code only when events arrive, without managing servers
Region	A geographic area containing a provider's data centres
Availability zone	An independent data centre (or group) within a region
Object storage	Files stored by name in buckets and used over HTTP
Port	A number that picks which program on a machine receives traffic
Virtual private cloud (VPC)	Your private network inside a cloud
Firewall	Rules that decide which network traffic may pass
Load balancer	The front door that spreads requests across healthy copies

Review Questions

1. Why is renting from the cloud usually cheaper than owning for a small app, even though the hourly price is higher?
2. Explain IaaS, PaaS, and serverless using the pizza picture.
3. What is a cold start, and which kind of service has it?
4. Why run copies of an app in two availability zones?
5. When would you use object storage instead of block storage?
6. In the lab, at roughly what traffic did the instance become cheaper than serverless?

You understand this when you can look at a new cloud service and say whether it is compute, storage, or networking, and what its meter counts.

Pipelines: From a Commit to Production

A pipeline turns “I changed some code” into “the change is running for users”, with checks at every step. This chapter covers continuous integration and delivery, stages and artifacts, environments, deployment strategies, and rollback. In the lab you build a tiny pipeline from a shell script.

The problem. Every release is a nervous afternoon of manual steps.

The question. How can a machine take my change from a commit to production the same safe way every time?

The Nervous Afternoon

A manual release means pulling the code, running the tests (if you remember), building, copying to the server, and restarting, all from memory. One forgotten step means a broken site on a Friday evening, so releases become rare, and each one bigger and scarier.

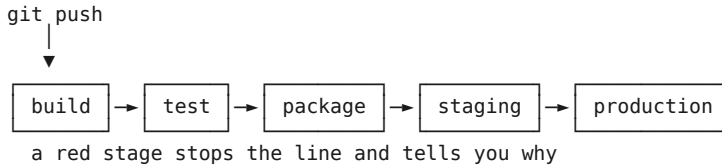
Factories solved this with the assembly line: every car passes the same stations in the same order, and an inspection station can stop the line when it finds a fault. A pipeline is an assembly line for code.

Just Enough Git

Pipelines start from git, the tool almost every team uses to track changes to code. A commit is one saved snapshot of a project, with a message such as “Fix the greeting” and an ID like 53ba61a.... The main line of history is usually called `main`. Pushing sends your commits to a shared copy of the project, on a service such as GitHub, and that push is the moment a pipeline wakes up.

The Pipeline

A **pipeline** is an automated sequence of stages that every change passes through. Each stage must succeed before the next one starts. If any stage fails, the line stops, and nobody gets the broken change.



A pipeline runs on a runner: a fresh container or VM that follows the pipeline’s instructions and is thrown away afterwards. Starting clean each time, it catches the “works on my machine” problems your laptop hides.

Continuous Integration

Continuous integration (CI) means that everyone merges their work into the shared branch often, at least daily, and that every change is built and tested automatically. The result is a simple signal: green (it passed) or red (it didn’t, and here is the log).

Good CI is fast: aim for minutes, or people start ignoring it.

Continuous Delivery and Continuous Deployment

After CI, the change has passed its tests. What happens next has two names that sound almost the same:

	Continuous delivery	Continuous deployment
After tests pass	Ready to release; a person presses a button	Released to production automatically
Good for	Teams that want a human decision	Teams with strong tests and quick rollback

Both are called CD, so people say CI/CD. With **continuous delivery**, every passing change *could* go live at any moment. With **continuous deployment**, it *does*. You will build continuous deployment in Chapter 10.

Artifacts: Build Once, Deploy Many

The package stage produces an **artifact**: the built, ready-to-run output, stored and labelled with a version. For container apps it is an image in a registry, tagged with the commit hash.

The rule is build once, deploy many: build the artifact one time and send that exact artifact to staging, then to production. A rebuild for production could pick up a newer library, and then staging tested something other than what production runs.

Keep old artifacts, too. They are what makes going back possible.

Environments in a Pipeline

The same artifact moves through the environments from Chapter 1, with a different configuration in each. Many teams move it to staging automatically, and to production only after a check: automatic tests on staging, or a person clicking “approve”.

Deployment Strategies

Deploying means swapping the running old version for the new one. There are four common ways, and each is a **deployment strategy**:

Strategy	How it works	Downtime	Extra cost	Risk
Recreate	Stop the old version, start the new one	A few seconds	None	Everyone gets a bad release at once
Rolling	Replace copies one at a time	None	Little	A bad release spreads gradually
Blue-green	Run the new version beside the old, then switch all traffic	None	Double, briefly	Switching back is instant
Canary	Send a small share of users to the new version first	None	Little	Only a few users meet a bad release

The canary is named after the birds miners carried underground as an early warning of bad air. A canary release lets a few per cent of users try the new version while you watch the error rate.

A **health check** is what makes all of these safe: a quick automatic test, such as calling the app’s /health address, that says whether a copy is working. A deployment should wait for the new version to report healthy before treating it as live. Chapter 9 adds one to a container.

Rollback

A **rollback** means going back to the previous good version. Because the old artifact is still in the registry, rolling back is just a deployment of an older version, and it should take seconds. The alternative is to roll forward: fix the bug and push a new version through the pipeline. Rolling back first and fixing calmly afterwards is usually the kinder option for your users.



Code rolls back easily; data doesn't. If the new version changed the database's structure, the old version may not understand it any more. Make database changes in small steps that both the old and new code can live with.

Lab: A Pipeline in Twenty Lines

You'll build a three-stage pipeline (test, build, deploy) as a shell script, watch a failing test stop the line, and roll back to an older artifact. The “server” is just a folder called staging.

1. Create a folder with a tiny module and its test:

```
mkdir -p ~/infra-labs/ch04 && cd ~/infra-labs/ch04
```

greet.py:

```
def greet(name):
    return f"Hello, {name}!"
```

test_greet.py:

```
import unittest

from greet import greet

class GreetTest(unittest.TestCase):
    def test_greet(self):
        self.assertEqual(greet("Asha"), "Hello, Asha!")

if __name__ == "__main__":
    unittest.main()
```

2. Create `deploy.sh`, which installs one artifact on the “server”:

```
#!/usr/bin/env bash
# Deploy one built artifact (a version) to the staging folder.
set -euo pipefail
VERSION="$1"

rm -rf staging && mkdir staging
tar -xzf "artifacts/greet-$VERSION.tar.gz" -C staging
echo "$VERSION" > staging/VERSION
echo "staging now runs version $VERSION"
```

And `pipeline.sh`, which runs the stages in order:

```
#!/usr/bin/env bash
# A tiny pipeline: test, build, deploy. Any failure stops the line.
set -euo pipefail
VERSION="$1"

echo "== Stage 1: test"
python3 -m unittest -q

echo "== Stage 2: build"
mkdir -p artifacts
tar -czf "artifacts/greet-$VERSION.tar.gz" greet.py
echo "built artifacts/greet-$VERSION.tar.gz"

echo "== Stage 3: deploy to staging"
./deploy.sh "$VERSION"
```

`set -euo pipefail` is the “stop the line” switch: the script exits at the first command that fails. `$1` is the first argument, the version.

3. Make both scripts executable and run the pipeline:

```
chmod +x pipeline.sh deploy.sh
./pipeline.sh 1.0.0

== Stage 1: test
-----
Ran 1 test in 0.000s

OK
== Stage 2: build
built artifacts/greet-1.0.0.tar.gz
== Stage 3: deploy to staging
staging now runs version 1.0.0
```

4. Break the code: in `greet.py`, change `Hello` to `Hi`. Run the pipeline for a new version,

then check what staging runs:

```
./pipeline.sh 1.1.0
cat staging/VERSION

== Stage 1: test
=====
FAIL: test_greet (test_greet.GreetTest.test_greet)
...
AssertionError: 'Hi, Asha!' != 'Hello, Asha!'
...
FAILED (failures=1)
1.0.0
```

The line stopped at stage 1. No 1.1.0 artifact was built, and staging still runs 1.0.0.

5. Change Hi back to Hello and run `./pipeline.sh 1.1.0` again. This time all three stages pass and the last line is staging now runs version 1.1.0.
6. Suppose users dislike 1.1.0. Roll back by deploying the older artifact, which is still on the shelf:

```
./deploy.sh 1.0.0
ls artifacts

staging now runs version 1.0.0
greet-1.0.0.tar.gz
greet-1.1.0.tar.gz
```

Expected results: a passing run goes through all three stages; a failing test stops the line before anything is built or deployed; and rollback is just `deploy.sh` with an older version, taking well under a second, because the old artifact was kept.

Summary, Key Terms, and Review Questions

Summary

- Manual releases are slow and risky because people forget steps. A pipeline runs the same stages, in the same order, for every change.
- CI builds and tests every change automatically. Continuous delivery makes every passing change ready to release; continuous deployment releases it.
- Build an artifact once, version it, and promote the same artifact through every environment.

- Recreate, rolling, blue-green, and canary deployments trade downtime, cost, and risk. Health checks make them safe.
- Rollback redeploys an older artifact. Database changes need extra care.

Key Terms

Term	Meaning
Pipeline	An automated sequence of stages every change passes through
Continuous integration (CI)	Building and testing every change automatically
Continuous delivery	Every passing change is ready to release at the press of a button
Continuous deployment	Every passing change is released automatically
Artifact	The built, versioned output of a pipeline
Deployment strategy	Recreate, rolling, blue-green, or canary: how the new version replaces the old
Health check	A quick automatic test of whether an app copy is working
Rollback	Returning to the previous good version

Review Questions

1. Why do manual releases tend to become rarer and riskier over time?
2. What is the difference between continuous delivery and continuous deployment?
3. Why should the artifact that reached staging be the one deployed to production, rather than a fresh build?
4. Compare blue-green and canary deployments. What does each protect you from?
5. Why is rolling back code easier than rolling back a database change?

You understand this when you can draw the pipeline for a small app, name what each stage produces, and explain how you would roll back if production broke after the last stage.

PART II

Use



What to ignore for now: orchestration at scale, multi-cloud, and writing your own CI runners.

Docker in Practice

This chapter turns Chapter 2's ideas into skills: write a Dockerfile for a small Python web app, build and run it, reach it through a port, keep its data in a volume, pass it configuration, make the image smaller and safer, and run it next to a database with Docker Compose.

The problem. I understand containers, but I've never built one.

The question. How do I describe my app to Docker, build it into an image, and run it the way a server would?

Meet tinyapp

The labs from here on use one very small web app, called `tinyapp`. It is written with FastAPI and has two addresses: `/health` says the app is alive and which version it is, and `/visits` counts visitors. It stores the count in a file, or in PostgreSQL when it is given a database address. Here is `main.py`:

```
"""tinyapp: a very small web app for the labs in this book."""
import os
from pathlib import Path

from fastapi import FastAPI

VERSION = os.environ.get("APP_VERSION", "dev")
DATABASE_URL = os.environ.get("DATABASE_URL", "")
DATA_FILE = Path(os.environ.get("DATA_DIR", "/tmp")) / "visits.txt"

app = FastAPI()

@app.get("/health")
def health():
    return {"status": "ok", "version": VERSION}
```

```

@app.get("/visits")
def visits():
    return {"visits": count_visit()}

def count_visit() -> int:
    if DATABASE_URL:
        return count_visit_in_postgres()
    count = int(DATA_FILE.read_text()) if DATA_FILE.exists() else 0
    DATA_FILE.write_text(str(count + 1))
    return count + 1

```

The file ends with `count_visit_in_postgres()`, a few lines that do the same count in a database table. Notice that every setting comes from an **environment variable**: a named value that the operating system hands to a process when it starts. `os.environ.get("APP_VERSION", "dev")` reads one, with a default. That is how one image can get a different configuration in each environment.

The app's libraries are listed in `requirements.txt`:

```

fastapi==0.141.1
uvicorn==0.54.0
psycpg[binary]==3.3.6

```

Uvicorn is the web server that runs a FastAPI app. Make a fresh folder, `~/infra-labs/ch05`, and copy into it `main.py`, `test_main.py`, and both `requirements` files from the book's `labs/tinyapp/` folder. The tests are for Chapter 6.

The Dockerfile

A **Dockerfile** is the recipe for an image: a text file of instructions that Docker follows from top to bottom, each one adding a layer. Create `Dockerfile` in the `ch05` folder:

```

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

```

```

RUN useradd --create-home appuser && mkdir /data && chown appuser /data
USER appuser
ENV DATA_DIR=/data

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Instruction	What it does here
FROM	Starts from the base image <code>python:3.12-slim</code> : Debian plus Python
WORKDIR	Makes <code>/app</code> the current folder for the following steps
COPY	Copies files from your folder into the image
RUN	Runs a command at build time, here <code>pip install</code> and creating a user
USER	Runs the app as <code>appuser</code> instead of the all-powerful root user
ENV	Sets an environment variable baked into the image
EXPOSE	Documents that the app listens on port 8000
CMD	The command that runs when a container starts

Two details matter. `requirements.txt` is copied and installed *before* `main.py`, which makes rebuilds fast (see below). And `--host 0.0.0.0` makes Uvicorn listen on all of the container's network interfaces. The default, `localhost`, means `localhost` only, and inside a container `localhost` is the container itself, so nothing outside could reach the app.

Next to it, create `.dockerignore`, the list of files never to send to Docker:

```

.git
.venv
__pycache__
.pytest_cache
.env
*.tar

```

Building and the Cache

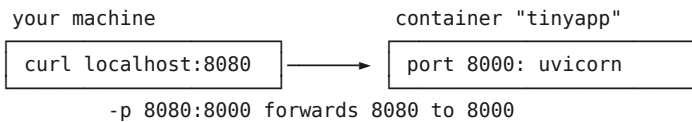
`docker build -t tinyapp:1.0 .` builds the image and names it `tinyapp:1.0`. The final `.` is the build context: the folder whose files `COPY` can see. Docker runs each instruction and saves a layer. When you rebuild, it reuses a layer from its build cache if the instruction and its input files haven't changed, and it rebuilds everything after the first change.

That is why the order matters. Your code changes all the time; your `requirements` rarely do. With `requirements` first, editing `main.py` rebuilds only the last few cheap layers, and the

slow `pip install` step says CACHED. Put `COPY . .` first and every one-line change reinstalls every library.

Running, Ports, and Logs

`docker run -d --name tinyapp -p 8080:8000 tinyapp:1.0` starts a container. `-d` runs it detached, in the background. `--name` gives it a name to use in other commands. `-p 8080:8000` is a port mapping: connections to port 8080 on your machine are forwarded to port 8000 inside the container.



A few commands you will use every day: `docker ps` lists running containers, `docker logs tinyapp` shows what the app printed, `docker exec tinyapp whoami` runs a command inside it, and `docker rm -f tinyapp` stops and removes it.

Volumes: Data That Survives

A container's own files vanish when it is removed, and the visit count lives in a file. A **volume** is storage that Docker manages outside any container, which you attach to a folder inside one. Replace the container, attach the same volume, and the data is still there. It's like parking the food truck but keeping the pantry in a warehouse.

`docker volume create tinyapp-data` creates one, and `-v tinyapp-data:/data` attaches it at `/data`, where the app writes its file. A bind mount is the other kind: `-v "$PWD":/work` attaches a folder from your own machine, which is handy in development, as in Chapter 1.

Configuration and Secrets

Pass configuration when the container starts, never when the image is built. `-e APP_VERSION=1.0` sets one environment variable; `--env-file .env` reads many from a file. The same image then runs in every environment with different settings.

A **secret** is configuration that must stay private: passwords, API keys, tokens. Secrets need extra care.



Never put a secret in a Dockerfile, with ENV or by copying a file. Anyone who can pull the image can read every layer, and `docker history` prints ENV values in plain text. Pass secrets at run time, keep `.env` files out of git and out of the image (that is why `.env` is in `.dockerignore`), and in production use your platform's secret store.

Smaller and Safer Images

Every megabyte is downloaded on every deploy, and every extra program is something an attacker might use. Some habits give you most of the benefit:

- *Start slim.* The full `python:3.12` image uses 1.62 GB of disk; `python:3.12-slim` uses 205 MB. Slim leaves out compilers and tools most apps never need.
- *Don't keep caches.* `pip install --no-cache-dir` stops pip saving downloads inside the image.
- *Ignore what you don't need.* `.dockerignore` keeps `.git`, virtual environments, and secrets out.
- *Don't run as root.* If an attacker breaks into your app, they get only what `appuser` can do.
- *Pin versions.* `python:3.12-slim` and `fastapi==0.141.1` rebuild the same way next month.

Larger projects also use multi-stage builds, where one stage compiles things and a second, clean stage receives only the results. `tinyapp` doesn't need one.

Compose: Several Containers Together

Real apps rarely run alone. `tinyapp` can use PostgreSQL, which would mean a network, a volume, a database container with the right settings, and the app container, started in the right order. Docker Compose lets you describe all of that in one file, `compose.yaml`, and start it with one command:

```
name: tinyapp

services:
  app:
    build: .
    ports:
      - "8080:8000"
    environment:
      APP_VERSION: "1.0"
      DATABASE_URL: postgresql://tinyapp:${DB_PASSWORD}@db:5432/tinyapp
    depends_on:
      db:
```

```

        condition: service_healthy

db:
  image: postgres:16-alpine
  environment:
    POSTGRES_USER: tinyapp
    POSTGRES_PASSWORD: ${DB_PASSWORD}
    POSTGRES_DB: tinyapp
  volumes:
    - db-data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD", "pg_isready", "-U", "tinyapp"]
    interval: 2s
    retries: 10

volumes:
  db-data:

```

name is the project's name, which Compose puts in front of everything it creates. Each entry under `services` becomes a container. Compose puts them on a private network where each one can reach the others by service name, which is why the app connects to `host db`. `depends_on` with `service_healthy` waits until PostgreSQL answers before starting the app. `${DB_PASSWORD}` is filled in from a file called `.env` in the same folder, so the password is not written in `compose.yaml`.

Lab: Build, Run, and Compose

1. In `~/infra-labs/ch05`, with `Dockerfile` and `.dockerignore` created as above, build the image. Docker draws a live progress display in the terminal; the steps are the same as these lines:

```

docker build -t tinyapp:1.0 .

#4 [1/6] FROM docker.io/library/python:3.12-slim@sha256:f77ac9e4...
...
#7 [3/6] COPY requirements.txt .
#8 [4/6] RUN pip install --no-cache-dir -r requirements.txt
#8 DONE 4.7s
#9 [5/6] COPY main.py .
#10 [6/6] RUN useradd --create-home appuser && mkdir /data && chown...
...
#11 naming to docker.io/library/tinyapp:1.0 done

```

2. Add a comment line to the end of `main.py` and build again as `tinyapp:1.1`. The install step is now #8 CACHED, and only steps 5 and 6 run. Remove the comment afterwards.
3. Run the app and call it:

```
docker run -d --name tinyapp -p 8080:8000 tinyapp:1.0
curl localhost:8080/health
curl localhost:8080/visits
curl localhost:8080/visits
docker exec tinyapp whoami
```

```
{"status":"ok","version":"dev"}
{"visits":1}
{"visits":2}
appuser
```

`docker run` also prints the container's long ID; from here on such lines are left out. `curl` doesn't end with a new line, so your prompt may appear right after the JSON.

4. Replace the container and call `/visits` again:

```
docker rm -f tinyapp
docker run -d --name tinyapp -p 8080:8000 tinyapp:1.0
curl localhost:8080/visits
```

```
{"visits":1}
```

The count was lost with the old container.

5. Now use a volume. Call `/visits` three times, replace the container, and call it once more:

```
docker rm -f tinyapp
docker volume create tinyapp-data
docker run -d --name tinyapp -p 8080:8000 -v tinyapp-data:/data tinyapp:1.0
curl localhost:8080/visits
curl localhost:8080/visits
curl localhost:8080/visits
docker rm -f tinyapp
docker run -d --name tinyapp -p 8080:8000 -v tinyapp-data:/data tinyapp:1.0
curl localhost:8080/visits
```

```
{"visits":1}
{"visits":2}
{"visits":3}
{"visits":4}
```

6. Pass configuration at run time:


```
docker rm -f tinyapp
docker run -d --name tinyapp -p 8080:8000 -e APP_VERSION=1.0 tinyapp:1.0
curl localhost:8080/health
```

```
{"status":"ok","version":"1.0"}
```

7. Remove it, then start the app with PostgreSQL. Create `.env` containing `DB_PASSWORD=change-me-locally`, save `compose.yaml` as shown, and run:

```
docker rm -f tinyapp
docker compose up -d

...
Network tinyapp_default Created
...
Volume tinyapp_db-data Created
...
Container tinyapp-db-1 Started
Container tinyapp-db-1 Healthy
Container tinyapp-app-1 Started
```

(Before these lines, Compose builds the image.) Wait two seconds for the app to start, then call `/visits` twice: you get 1, then 2.

8. Stop everything and start it again. The containers and network are removed, but the database volume stays:

```
docker compose down
docker compose up -d
curl localhost:8080/visits

{"visits":3}
```

9. Clean up. `-v` also deletes the database volume:

```
docker compose down -v
docker volume rm tinyapp-data
```

Expected results: the second build reuses the cached install layer; the app runs as `appuser`; the count resets when the container is replaced without a volume and survives with one; `APP_VERSION` changes without rebuilding; and with Compose the count lives in PostgreSQL and survives down and up.

Summary, Key Terms, and Review Questions

Summary

- A Dockerfile is an image's recipe. Each instruction adds a layer; copy requirements before code so rebuilds are fast.
- `docker run -d -p 8080:8000` runs a container in the background and forwards a port. Apps in containers must listen on `0.0.0.0`.
- Volumes keep data when containers are replaced.
- Configuration arrives as environment variables at run time. Secrets never go into images.
- Slim bases, no caches, `.dockerignore`, a non-root user, and pinned versions make images smaller and safer.
- Docker Compose describes several containers, their network, and their volumes in one file.

Key Terms

Term	Meaning
Environment variable	A named value given to a process when it starts
Dockerfile	The text recipe Docker follows to build an image
Volume	Docker-managed storage that outlives containers
Secret	Private configuration such as a password or key

Review Questions

1. Why does the Dockerfile copy `requirements.txt` before `main.py`?
2. What does `-p 8080:8000` do, and why must the app listen on `0.0.0.0`?
3. Why should a password never be set with `ENV` in a Dockerfile?
4. Name four habits that make an image smaller or safer.
5. How does the app container find the database in the Compose file?

You understand this when you can write a Dockerfile for a small Python app from memory, run it with a port and a volume, and explain what survives when the container is replaced.

Your First Pipeline

In this chapter you write a real CI pipeline with GitHub Actions: it runs the tests on every push, builds the image, and publishes it to a registry, using a secret on the way. You will run it on your own machine with `act`, so you need no GitHub account.

The problem. I keep forgetting to run the tests before I push.

The question. How do I make a machine run my checks on every push, so that forgetting is no longer possible?

GitHub Actions in One Picture

GitHub Actions is the pipeline service built into GitHub. You describe a pipeline in a YAML file inside your repository, and GitHub runs it for you whenever something happens. Think of it as a robot assistant that watches the repository: “whenever someone pushes to `main`, do these things, in this order, and tell me if anything fails.”

Five words cover almost everything:

Word	Meaning	In the file
Workflow	One pipeline, in one YAML file under <code>.github/workflows/</code>	the whole file
Event	What starts the workflow, such as a push	<code>on:</code>
Job	A group of steps that runs on one fresh runner	<code>jobs:</code>
Step	One command, or one action	<code>steps:</code>
Action	A reusable, ready-made step someone else wrote	<code>uses:</code>

```

workflow (.github/workflows/ci.yml, started by a push)
├── job: test (runner 1)
│   ├── step: check out the code
│   ├── step: set up Python
│   ├── step: pip install
│   └── step: pytest
└── job: publish (runner 2, only after "test" passes)
    ├── step: build the image
    └── step: push the image

```

Jobs run at the same time unless one says `needs: another`. Steps inside a job run in order, and the first failing step fails the job.

YAML is the file format. It is Python-like in one important way: indentation matters. `key:` value sets a value, an indented block belongs to the line above it, and lines starting with `-` form a list.

Running Workflows Locally with act

On GitHub, each job runs on a fresh virtual machine. `act` imitates that on your computer: it reads your workflow files and runs each job in a Docker container that looks like GitHub's runner. The image it uses, `catthehacker/ubuntu:act-latest`, is over 2 GB, so the first run takes a while to download it.

Put a file called `.actrc` in your repository. `act` reads its options from there, so you don't have to type them each time:

```

-P ubuntu-latest=catthehacker/ubuntu:act-latest
--pull=false

```

The first line picks the image for `ubuntu-latest` jobs; the second stops `act` checking for a newer one on every run.

To build and push images, a job needs Docker. `act` shares yours with each job by attaching the Docker socket, the file through which the `docker` command talks to the Docker daemon. So a `docker build` inside a job builds on your own Docker, and the image appears in your own `docker images`.



If `act` can't reach your Docker, point it at the socket with `--container-daemon-socket unix:///path/to/docker.sock`.

A Workflow That Runs the Tests

tinyapp has two small tests in `test_main.py`. They call the app in memory, without a server:

```
from fastapi.testclient import TestClient

import main

client = TestClient(main.app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"

def test_visits_count_up(tmp_path, monkeypatch):
    monkeypatch.setattr(main, "DATA_FILE", tmp_path / "visits.txt")
    assert client.get("/visits").json() == {"visits": 1}
    assert client.get("/visits").json() == {"visits": 2}
```

The first half of the workflow runs them on every push to main:

```
name: CI

on:
  push:
    branches: [main]
    pull_request:

env:
  IMAGE: localhost:5001/tinyapp

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements-dev.txt
      - run: pytest -q
```

on: lists the events: pushes to main, and pull requests. runs-on: picks the runner. actions/checkout copies the repository into the runner, and actions/setup-python installs the Python version you ask for. requirements-dev.txt adds the test tools to the app's own requirements:

```
-r requirements.txt
pytest==9.1.1
httpx2==2.13.1
```

The first line pulls in requirements.txt; httpx2 is the HTTP client FastAPI's test client needs. The env: block sets a variable every job can use.

Building and Publishing the Image

Once the tests pass, the second job builds the image and pushes it to a registry. You'll run your own registry: Docker's registry:2 image is a complete registry in one container.

```
publish:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Check the registry token
      env:
        REGISTRY_TOKEN: ${ secrets.REGISTRY_TOKEN }
      run: |
        test -n "$REGISTRY_TOKEN" || { echo "REGISTRY_TOKEN is missing"; exit 1;
↪ }
        echo "Using token $REGISTRY_TOKEN"
    - name: Build the image
      run: docker build -t $IMAGE:${ github.sha } .
    - name: Push the image
      run: docker push $IMAGE:${ github.sha }
```

needs: test means “only if the test job passed”. \${ github.sha } is filled in with the commit hash, so every image is tagged with the exact commit it was built from. That is the build-once artifact from Chapter 4.



The registry listens on port 5001, not the usual 5000, because on macOS port 5000 is taken by the AirPlay Receiver. Docker trusts registries on localhost without HTTPS, which is fine for a lab and never for anything real.

Secrets in CI

Pipelines need secrets, such as a registry password. They never go in the workflow file, which is in git for everyone to read.

On GitHub, you store secrets in the repository's settings (Settings, then Secrets and variables, then Actions). A workflow reads one as `${{ secrets.REGISTRY_TOKEN }}`, and GitHub replaces the value with `***` anywhere it would appear in the logs. `act` reads secrets from a file called `.secrets` in the same `NAME=value` format, and masks them the same way.

Our “Check the registry token” step stands in for `docker login`: it fails if the secret is missing, and prints it only so you can watch the masking. Never print a secret in a real workflow.



Add `.secrets` and `.env` to `.gitignore` before your first commit. Masking protects the logs, not the repository, and it can't stop a step from sending the secret somewhere else. Only use actions you trust in workflows that can see secrets.

Lab: Tests on Every Push

1. Start the local registry and check that it is empty:

```
docker run -d --name registry -p 5001:5000 registry:2
curl localhost:5001/v2/_catalog
```

```
{"repositories":[]}
```

2. Make a fresh folder, `~/infra-labs/ch06`, and copy in the app's four files (from `labs/tinyapp/`) and Chapter 5's `Dockerfile` and `.dockerignore` (from `labs/ch05/` and `labs/tinyapp/`). Then create `.gitignore` (with `.env`, `.secrets`, `__pycache__`, `.pytest_cache`, and `.venv`), `.actrc` as above, `.secrets` containing `REGISTRY_TOKEN=not-a-real-token-123`, and `.github/workflows/ci.yml` with both jobs. Then make it a git repository and commit:

```
git init -b main
git add -A
git commit -m "tinyapp with CI"
```

If git asks who you are, set `user.name` and `user.email` with `git config` inside the folder, then commit again.

3. List the jobs `act` found:

```
act -l
```

Stage	Job ID	Job name	Workflow name	Workflow file	Events
0	test	test	CI	ci.yml	push,pull_request
1	publish	publish	CI	ci.yml	push,pull_request

4. Run the workflow as if you had pushed. The first run downloads the runner image and the actions:

```
act push

[CI/test] | 2 passed in 0.13s
...
[CI/test] 🟩 Job succeeded
...
[CI/publish] | Using token ***
...
[CI/publish] | 53ba61aa33f5...: digest: sha256:cdc5d934... size: 856
[CI/publish] 🟢 Success - Main Push the image [736.652791ms]
...
[CI/publish] 🟩 Job succeeded
```

5. Break a test on purpose: in `main.py`, change `"status": "ok"` to `"status": "OK"`, commit, and run `act` again:

```
git commit -am "Shout the status"
act push

...
[CI/test] | FAILED test_main.py::test_health - AssertionError: assert 'OK'
↳ == 'ok'
[CI/test] | 1 failed, 1 passed in 0.13s
[CI/test] 🛑 Failure - Main pytest -q [334.395125ms]
[CI/test] 🟩 Job failed
Error: Job 'test' failed
```

The publish job never ran, so no broken image reached the registry.

6. Undo the change with a new commit, then run once without the secrets file to see the check fail:

```
git revert --no-edit HEAD
mv .secrets ../secrets.bak
act push
mv ../secrets.bak .secrets
```



```
...
[CI/publish] | REGISTRY_TOKEN is missing
[CI/publish] ❌ Job failed
Error: Job 'publish' failed
```

7. Run `act push` once more. Both jobs succeed. Ask the registry what it holds:

```
curl localhost:5001/v2/tinyapp/tags/list

{"name":"tinyapp","tags":["53ba61aa33f5...", "10ff8b9d..."]}
```

Expected results: a clean push runs the tests and publishes an image tagged with the commit hash; a failing test stops the workflow before anything is built; a missing secret stops the publish job; and the secret shows as `***` in logs. Leave the registry running for the next chapters. Your commit hashes will differ.

Summary, Key Terms, and Review Questions

Summary

- A GitHub Actions workflow is a YAML file of jobs and steps, started by an event.
- Jobs run on fresh runners; `needs:` orders them; a failing step stops the line.
- Tag images with the commit hash. Secrets live in the CI system and are masked in logs.

Key Terms

Term	Meaning
Workflow	One pipeline, defined in a YAML file in <code>.github/workflows/</code>

Review Questions

1. What is the difference between a job and a step?
2. What does `needs: test` do, and what would happen without it?
3. Why tag the image with `${{ github.sha }}` rather than `latest`?
4. Where should a CI secret live, and what does masking protect against?

You understand this when you can read a workflow and say what runs, when, and what a failing test stops.

Cloud Building Blocks

A cloud provider's menu is enormous, but a small app uses only a handful of dishes: accounts and access control, virtual machines, object storage, managed databases, firewalls, serverless functions, and cost alerts. In the lab you run an S3-compatible object store locally and give an app a key that can do only what it needs.

The problem. The cloud console has 200 services, and I need three.

The question. Which few building blocks does a small app actually need, and how do I use them safely?

The Few You Actually Need

A console is the provider's website for managing things by clicking, and it lists hundreds of services. For a small web app, this short list covers nearly everything:

Need	Building block
Who may do what	Accounts and IAM
Somewhere to run code	A virtual machine, a container service, or functions
Somewhere to keep files	Object storage
Somewhere to keep records	A managed database
Control over who can connect	Firewalls
No surprises on the bill	Budgets and cost alerts

Accounts and IAM

When you sign up, you get an account: the container for everything you create and pay for. The login you signed up with is the **root user**, and it can do anything, including closing the

account. Protect it with multi-factor authentication (MFA), a second proof such as a code from your phone, and don't use it for daily work.

Identity and access management (IAM) decides who may do what. Picture a hotel key card that opens your room and the gym, but not other rooms or the safe. IAM hands out key cards:

- An IAM user is a person or program with long-term credentials.
- A **role** is a set of permissions that a machine or service takes on, receiving short-lived credentials automatically. An app on a cloud VM should use a role, not a stored password.
- A **policy** is a document listing what is allowed: which actions, on which resources.

Here is a policy that allows reading and writing objects in one bucket, called `uploads`, and nothing else:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": ["arn:aws:s3:::uploads/*"]
    }
  ]
}
```

Anything not explicitly allowed is denied. `arn:aws:s3:::uploads/*` is how AWS names “every object in the bucket `uploads`”.

The rule behind good IAM is **least privilege**: give each person and program only the permissions it needs. If the app's key leaks, the thief can touch only the app's own bucket.

Programs that can't use a role sign their requests with an **access key**: a pair of a key ID (like a username) and a secret key (like a password).



Leaked access keys are the most common way cloud accounts get abused. Never commit them to git, never put them in an image, and prefer roles, whose credentials expire on their own.

Virtual Machines

Launching a VM means choosing a region, a disk image (such as Ubuntu Linux), an instance type, firewall rules, and an SSH key, which proves it's you when you open a terminal on it. A minute later you can log in, install Docker, and run your container as on your laptop. A *stopped* VM still pays for its disk; only a *terminated* (deleted) one costs nothing.

Object Storage

New buckets are private: only identities with permission can read them. Keep it that way; a bucket accidentally made public is one of the most common causes of data leaks.

Many products speak S3's interface, including MinIO, a storage server you can run yourself. In the lab, MinIO plays the cloud's object storage, and `boto3`, the official AWS library for Python, talks to it exactly as it would to S3.



MinIO no longer publishes ready-made images for its free edition, so the lab uses free builds of its open-source code from Chainguard: `cgr.dev/chainguard/minio` and `cgr.dev/chainguard/minio-client`.

Managed Databases

A **managed database** is a database, such as PostgreSQL, that the provider installs, updates, backs up, and can copy to a second availability zone. You connect with a host name, port, user, and password as usual. It costs more than running PostgreSQL yourself, and for almost every small team it's worth it. Keep it in a private subnet, reachable only by the app.

Firewalls

In the cloud, firewall rules are usually called security groups: lists of what may come in, attached to machines or databases. Everything not listed is blocked.

Rule	Why
Allow port 443 from anywhere to the load balancer	Users reach the app over HTTPS
Allow port 8000 from the load balancer to the app	Only the front door talks to the app
Allow port 5432 from the app to the database	Only the app talks to the database
Allow port 22 (SSH) from your own IP address only	Nobody else can even try to log in



“Allow 0.0.0.0/0” means “allow the whole internet”. Automated scanners find open database and SSH ports within minutes of them appearing.

Serverless Functions

A function service runs your code whenever an event happens: an HTTP request, a file landing in a bucket, a nightly timer. There is nothing to patch and nothing to pay while it's idle,

but functions have a maximum run time (often 15 minutes) and cold starts. They shine for glue work, such as resizing each uploaded image.

Cost Alerts

Before you create anything in a real account, create a budget alert: in the billing section, set a monthly amount, say \$10, and ask for an email at 50%, 80%, and 100% of it, or when the forecast says you'll pass it. An alert doesn't stop spending; it tells you in time to stop it yourself. Label what you create, so the bill shows what each thing costs, and delete experiments when you're done.

Lab: Least Privilege with Local Object Storage

You'll run MinIO, create two buckets, and give an "app" an access key that can read and write one bucket only. Then you'll prove what it can and cannot do.

1. Create a folder and a private network, and start MinIO with a volume for its data:

```
mkdir -p ~/infra-labs/ch07 && cd ~/infra-labs/ch07
docker network create cloudlab
docker run -d --name minio --network cloudlab -p 9000:9000 \
-v minio-data:/data \
-e MINIO_ROOT_USER=admin -e MINIO_ROOT_PASSWORD=change-me-please \
cgr.dev/chainguard/minio server /data
```

admin is MinIO's root user, like the cloud account's owner.

2. The MinIO client, mc, is the admin tool. It runs in a container on the same network, so define a small shell function that runs it for you. MC_HOST_local tells it where the server is and how to log in, and "\$@" passes along whatever you type after mc:

```
mc() {
  docker run --rm --network cloudlab -v "$PWD":/work -w /work \
  -e MC_HOST_local=http://admin:change-me-please@minio:9000 \
  cgr.dev/chainguard/minio-client "$@"
}
```

3. Create two buckets, and put a "private" file in one of them:

```
printf 'name,amount\nAsha,1200\n' > report.csv
mc mb local/uploads
mc mb local/billing
mc cp report.csv local/billing/
mc ls local

Bucket created successfully `local/uploads`.
Bucket created successfully `local/billing`.
`/work/report.csv` -> `local/billing/report.csv`
...
[2026-09-26 14:38:42 UTC]      0B billing/
[2026-09-26 14:38:42 UTC]      0B uploads/
```

4. Save the policy from “Accounts and IAM” as uploads-only.json. Create it in MinIO, create a user for the app, and attach the policy:

```
mc admin policy create local uploads-only uploads-only.json
mc admin user add local app-uploader uploader-secret-123
mc admin policy attach local uploads-only --user app-uploader

Created policy `uploads-only` successfully.
Added user `app-uploader` successfully.
Attached Policies: [uploads-only]
To User: app-uploader
```

In MinIO, the user name and password serve as the access key ID and secret key.

5. Create a virtual environment, install boto3, and save this as s3_demo.py:

```
python3 -m venv .venv && . .venv/bin/activate && pip install boto3==1.43.103

"""Use object storage the way an app would: with a limited access key."""
import os

import boto3
from botocore.exceptions import ClientError

s3 = boto3.client(
    "s3",
    endpoint_url="http://localhost:9000",
    aws_access_key_id=os.environ["S3_ACCESS_KEY"],
    aws_secret_access_key=os.environ["S3_SECRET_KEY"],
    region_name="us-east-1",
)

def read(bucket, key):
```

```

return s3.get_object(Bucket=bucket, Key=key)["Body"].read().decode()

def attempt(label, action):
    try:
        result = action()
    except ClientError as error:
        print(f"DENIED {label}: {error.response['Error']['Code']}")
        return
    print(f"OK {label}", result if isinstance(result, (str, list)) else
        ↪ "")

attempt("upload uploads/hello.txt",
        lambda: s3.put_object(Bucket="uploads", Key="hello.txt", Body=b"hi!"))
attempt("read uploads/hello.txt", lambda: read("uploads", "hello.txt"))
attempt("read billing/report.csv", lambda: read("billing", "report.csv"))
attempt("delete uploads/hello.txt",
        lambda: s3.delete_object(Bucket="uploads", Key="hello.txt"))
attempt("list all buckets",
        lambda: [bucket["Name"] for bucket in s3.list_buckets()["Buckets"]])

```

6. Run it with the app's limited key:

```

S3_ACCESS_KEY=app-uploader S3_SECRET_KEY=uploader-secret-123 python3
↪ s3_demo.py

```

```

OK      upload uploads/hello.txt
OK      read uploads/hello.txt hi!
DENIED  read billing/report.csv: AccessDenied
DENIED  delete uploads/hello.txt: AccessDenied
DENIED  list all buckets: AccessDenied

```

7. Run it again with the root credentials (S3_ACCESS_KEY=admin S3_SECRET_KEY=change-me-please). Every line now says OK: the root user reads the report, deletes the upload, and lists ['billing', 'uploads'].

8. Try reading the private file with no credentials at all:

```

curl -s -o /dev/null -w '%{http_code}\n'
↪ http://localhost:9000/billing/report.csv

```

403

9. Clean up:

```
docker rm -f minio && docker volume rm minio-data && docker network rm
↪ cloudlab
```

Expected results: the app’s key can put and get objects in uploads, and nothing else, not even deleting its own upload or listing buckets; the root user can do everything; and an anonymous request gets 403 Forbidden, because buckets are private by default.

Summary, Key Terms, and Review Questions

Summary

- A small app needs identity, compute, storage, a database, firewalls, and a budget.
- Lock the root user away with MFA. Policies decide who may do what; prefer roles to stored keys, and grant least privilege.
- A VM costs money until it is terminated; buckets are private by default; managed databases buy you tested backups.
- Firewalls block everything not listed. Never open databases or SSH to the whole internet. Budget alerts warn you before the bill does.

Key Terms

Term	Meaning
Root user	An account’s all-powerful owner login
Identity and access management (IAM)	The service that decides who may do what
Role	Permissions taken on with short-lived credentials
Policy	A document listing allowed actions on resources
Least privilege	Giving each identity only the permissions it needs
Access key	A key ID and secret a program signs requests with
Managed database	A database the provider runs and backs up

Review Questions

1. Why should you not use the root user for everyday work?
2. How do an IAM user and a role differ, and which should an app on a VM use?
3. Write the firewall rules for an app behind a load balancer, with a database.
4. Does a budget alert stop spending? What should you do when one arrives?

You understand this when you can write a policy that gives a program exactly one bucket.

Infrastructure as Code

Your app is packaged and your pipeline automated. The last manual part is the infrastructure itself. This chapter describes it in files that Terraform reads and builds: providers, resources, plan, apply, and state. In the lab, Terraform creates, changes, repairs, and deletes a container.

The problem. I clicked 40 buttons to build this, and I can't do it again.

The question. How do I write my infrastructure down so that a tool can build it the same way every time?

Clicks Don't Repeat

Clicking through a console is fine for exploring and bad for anything you need to keep. Nobody remembers the 40 clicks or can review them, and when you need a second copy for staging, you start from zero.

Builders don't construct a house from memory. They work from a blueprint that anyone can read, check, copy, and correct. **Infrastructure as code** (IaC) is the blueprint approach: you describe your servers, networks, databases, and permissions in text files, keep those files in git next to your app, and let a tool create exactly what they describe. You get:

- *Repeatability.* Staging and production are built from the same files.
- *Review.* A change to the infrastructure is a change to a file, so a colleague can read it before it happens.
- *History.* Git shows who changed what, when, and why.
- *Disposability.* If you can rebuild it in minutes, you can also delete it without fear.

Declarative: Say What, Not How

You can tell a taxi driver “take me to the station”, or you can give every turn yourself. The first is **declarative**: you describe the destination and let the expert find the route. The second is imperative: you list the steps.

Terraform is declarative: you write “there should be a container like this”, and it compares that with what exists and makes only the changes needed. Run it twice and the second run does nothing, because nothing differs.

Terraform

Terraform is the most widely used IaC tool. You write its files in HCL (HashiCorp Configuration Language), a simple language of blocks and `name = value` settings, in files ending in `.tf`.

Terraform itself knows nothing about clouds. A **provider** is a plugin that knows how to talk to one system’s API: AWS, Azure, Google Cloud, GitHub, or, for this book’s labs, your local Docker. A **resource** is one thing a provider manages, such as a VM, a bucket, or a container.

Here is a complete configuration that runs the nginx web server in a container:

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 4.6"
    }
  }
}

provider "docker" {}

resource "docker_image" "web" {
  name          = "nginx:alpine"
  keep_locally = true
}

resource "docker_container" "web" {
  name = "tf-web"
  image = docker_image.web.image_id

  ports {
    internal = 80
    external = 8081
  }
}
```

```

    }
  }
}

```

The `terraform` block says which provider to download: `kreuzwerker/docker`, any 4.x version from 4.6. The `provider` block configures it; empty means “use the local Docker”. Each resource block has a type (`docker_container`) and a name you choose (`web`).

The line `image = docker_image.web.image_id` is a reference: the container uses an attribute of the image resource. References tell Terraform the order: the image must exist before the container. You never write the order yourself.

`keep_locally = true` stops Terraform deleting the image from your machine on destroy.

The Workflow: init, plan, apply, destroy

Command	What it does
<code>terraform init</code>	Downloads the providers and prepares the folder. Run once, and after changing providers
<code>terraform plan</code>	Compares the files with reality and shows what would change. Changes nothing
<code>terraform apply</code>	Shows the plan again, asks for yes, then makes the changes
<code>terraform destroy</code>	Deletes everything this configuration manages, after asking

A **plan** marks each resource with a symbol: `+` create, `~` update in place, `-` destroy, and `-/+` destroy and re-create, which Terraform does when a setting can’t be changed on a live resource. Read the plan every time. It is your last chance to catch “1 to destroy” before it happens.

State

After `apply`, Terraform writes a file called `terraform.tfstate`. This **state** is Terraform’s notebook: which real object belongs to which resource block, with its ID and attributes. On the next run, Terraform reads the notebook, asks the provider what really exists, and compares both with your files.

If someone changes or deletes something by hand, reality no longer matches the files. That difference is called **drift**, and the next `plan` shows it and offers to put things back.



State can contain secrets, such as database passwords, in plain text. Never commit `terraform.tfstate` to git, and never edit it by hand. Teams keep it in remote state: a shared, access-controlled location, such as a storage bucket, with a lock so two people can't apply at once.

Lab: Your First Terraform

1. Create `~/infra-labs/ch08/main.tf` with the configuration above, then initialise the folder:

```
cd ~/infra-labs/ch08
terraform init
```

```
Initializing the backend...
```

```
Initializing provider plugins...
```

```
- Finding kreuzwerker/docker versions matching "~> 4.6"...
```

```
- Installing kreuzwerker/docker v4.6.0...
```

```
- Installed kreuzwerker/docker v4.6.0 (self-signed, key ID 0DCE698927DAF8EC)
```

```
...
```

```
Terraform has been successfully initialized!
```

`init` also writes `.terraform.lock.hcl`, which pins the exact provider version. Commit it.

2. Check the file and preview the changes:

```
terraform validate
terraform plan
```

```
Success! The configuration is valid.
```

```
...
```

```
# docker_container.web will be created
```

```
+ resource "docker_container" "web" {
```

```
  + name                               = "tf-web"
```

```
...
```

```
# docker_image.web will be created
```

```
+ resource "docker_image" "web" {
```

```
...
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
```

3. Apply it. Terraform shows the plan again and asks for confirmation; type yes. The first time, it also downloads the nginx image:

```
terraform apply
```

```
...
  Enter a value: yes

docker_image.web: Creating...
docker_image.web: Creation complete after 0s
  ↳ [id=sha256:1ed1b0e1d765...nginx:alpine]
docker_container.web: Creating...
docker_container.web: Creation complete after 1s [id=c5c48da6fc2e...]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

4. Check the result and the state:

```
curl -s localhost:8081 | grep title
terraform state list

<title>Welcome to nginx!</title>
docker_container.web
docker_image.web
```

5. Run terraform plan again. Nothing differs, so nothing would change:

```
No changes. Your infrastructure matches the configuration.
...
```

6. Change external = 8081 to 8082 in main.tf, and plan:

```
# docker_container.web must be replaced
-/+ resource "docker_container" "web" {
...
    ~ external = 8081 -> 8082 # forces replacement
...
Plan: 1 to add, 0 to change, 1 to destroy.
```

A running container's ports can't be changed, so Terraform will replace it. Run `terraform apply` and type yes.

7. Cause drift by deleting the container behind Terraform's back, then plan and apply:

```
docker rm -f tf-web
terraform plan
```

```
# docker_container.web will be created
...
Plan: 1 to add, 0 to change, 0 to destroy.
```

After terraform apply, curl localhost:8082 answers again.

8. Delete everything, typing yes when asked:

```
terraform destroy

Plan: 0 to add, 0 to change, 2 to destroy.
...
docker_container.web: Destruction complete after 0s
docker_image.web: Destroying... [id=sha256:1ed1b0e1d765...nginx:alpine]
docker_image.web: Destruction complete after 0s

Destroy complete! Resources: 2 destroyed.
```

Expected results: plan previews without changing anything; apply creates the image and container; a second plan finds no changes; changing the port forces a replacement; a container deleted by hand is detected and re-created; and destroy removes it all (the nginx image stays on disk because of keep_locally).

Summary, Key Terms, and Review Questions

Summary

- Infrastructure as code describes infrastructure in files kept in git, so it can be rebuilt, reviewed, and versioned.
- Terraform is declarative: you describe the result, and it works out the changes.
- Providers talk to systems; resources are the things they manage; references set the order.
- init, plan, apply, and destroy are the whole workflow. Always read the plan.
- State records what Terraform built. Drift is reality differing from the files. State can hold secrets, so keep it out of git.

Key Terms

Term	Meaning
Infrastructure as code (IaC)	Describing infrastructure in files that a tool builds from
Declarative	Describing the result you want, not the steps to get there
Provider	A Terraform plugin that manages one system through its API
Resource	One thing a provider manages
Plan	Terraform's preview of the changes it would make
State	Terraform's record of what it built
Drift	Differences between reality and the configuration

Review Questions

1. Name three problems with building infrastructure by clicking in a console.
2. What does “declarative” mean, and why does running `terraform apply` twice change nothing the second time?
3. How does Terraform know to create the image before the container?
4. What do `+`, `~`, and `-/+` mean in a plan?
5. Why must `terraform.tfstate` stay out of git?

You understand this when you can read a Terraform plan and predict exactly what will be created, replaced, or destroyed before you type yes.

PART III

Apply



What to ignore for now: tuning, autoscaling policies, and production-grade monitoring.

Lab: Ship a Small Python App in a Container

This whole chapter is a lab. You take `tinyapp` from source code to a versioned image that knows how to report its own health, watch a healthy and an unhealthy container side by side, publish the image to a registry, and run it from there on a machine that doesn't have it yet.

The problem. I have a working app and want it to run anywhere.

The question. What does it take to turn my app into one tested image that any machine can pull and run?

What You'll Build

```

source → tests → docker build → tinyapp:1.0.0
                                |
                                docker run: healthy?
                                |
                                docker push → localhost:5001 registry
                                |
                                "another machine" ← docker pull + docker run
  
```

Start in a fresh folder, `~/infra-labs/ch09`, holding the app's four files and Chapter 5's `Dockerfile` and `.dockerignore`, copied from the book's `labs/` folder. You also need the registry from Chapter 6: if it is stopped, `docker start registry` starts it again, and if you removed it, run `docker run -d --name registry -p 5001:5000 registry:2`. If a container called `tinyapp` is still running from an earlier chapter, remove it with `docker rm -f tinyapp`.

Step 1: Test the App Before You Package It

An image is only as good as the code inside it, so run the tests first. In `~/infra-labs/ch09:`

```
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements-dev.txt
pytest -q

..
2 passed in 0.29s [100%]
```

`.venv` is listed in `.dockerignore`, so the virtual environment never ends up in the image.

Step 2: Teach the Image to Report Its Health

A running process is not the same as a working app. The process can be alive while the app is stuck, misconfigured, or listening on the wrong port. A health check asks the app itself.

Docker supports this with the `HEALTHCHECK` instruction: a command that Docker runs inside the container every few seconds. If it succeeds (exit code 0), the container is healthy; after a few failures in a row, it is unhealthy. Add two lines to the `Dockerfile`, just before `CMD`:

```
HEALTHCHECK --interval=5s --timeout=3s --start-period=5s --retries=3 \
  CMD python -c "import urllib.request;
    ↳ urllib.request.urlopen('http://localhost:8000/health', timeout=2)"
```

Option	Meaning
<code>--interval=5s</code>	Check every 5 seconds
<code>--timeout=3s</code>	A check that takes longer than 3 seconds counts as failed
<code>--start-period=5s</code>	Failures in the first 5 seconds don't count; the app is starting
<code>--retries=3</code>	Three failures in a row mark the container unhealthy

The check uses Python's own `urllib`, because the slim image doesn't include `curl`. `urlopen` raises an error on a failed connection or an error status, which makes Python exit with code 1. Real services use longer intervals, such as 30 seconds; 5 seconds keeps the lab quick.

Step 3: Build a Versioned Image

Give each release a version. A common scheme is **semantic versioning**. A version has three numbers, MAJOR.MINOR.PATCH: raise PATCH for bug fixes, MINOR for new features, and MAJOR for changes that break something for users. This is version 1.0.0:

```
docker build -t tinyapp:1.0.0 .

...
#7 [4/6] RUN pip install --no-cache-dir -r requirements.txt
...
#10 [6/6] RUN useradd --create-home appuser && mkdir /data && chown...
#11 naming to docker.io/library/tinyapp:1.0.0 done
```

Step 4: Run It and Watch It Become Healthy

```
docker run -d --name tinyapp -p 8080:8000 -e APP_VERSION=1.0.0 tinyapp:1.0.0
docker ps --format 'table {{.Names}}\t{{.Status}}'
```

```
NAMES      STATUS
tinyapp    Up Less than a second (health: starting)
```

Wait about seven seconds and ask again:

```
docker ps --format 'table {{.Names}}\t{{.Status}}'
docker inspect --format '{{.State.Health.Status}}' tinyapp
curl localhost:8080/health
```

```
NAMES      STATUS
tinyapp    Up 7 seconds (healthy)
healthy
{"status":"ok","version":"1.0.0"}
```

`docker inspect` prints everything Docker knows about a container as JSON; `--format` picks out one field. Scripts use exactly this command to wait for a new version to become healthy, as you'll see in Chapter 10.

Step 5: See What Unhealthy Looks Like

Start a second container from the same image, but override its command so the app listens on port 9999 instead of 8000. The process runs perfectly well. It just isn't where the health check (or any user) expects it:

```
docker run -d --name tinyapp-broken tinyapp:1.0.0 uvicorn main:app --port 9999
```

Wait about 20 seconds, then look:

```
docker ps --filter name=tinyapp-broken --format 'table {{.Names}}\t{{.Status}}'
docker inspect --format '{{(index .State.Health.Log 0).Output}}' tinyapp-broken |
  ↪ grep URLError

NAMES          STATUS
tinyapp-broken Up 22 seconds (unhealthy)
               raise URLError(err)
urllib.error.URLError: <urlopen error [Errno 111] Connection refused>
```

Docker keeps the output of the last few checks, and here it says exactly what went wrong: nothing answered on port 8000. Without a health check, this container would look fine in `docker ps`. Remove it:

```
docker rm -f tinyapp-broken
```



On its own, Docker only *reports* an unhealthy container; it doesn't restart it. Platforms built on containers, such as cloud container services and Kubernetes, act on health checks: they stop sending traffic to unhealthy copies and replace them. Chapter 12 returns to this.

Step 6: Push the Image to the Registry

To push, the image's name must start with the registry's address. `docker tag` adds a second name to the same image; nothing is copied:

```
docker tag tinyapp:1.0.0 localhost:5001/tinyapp:1.0.0
docker push localhost:5001/tinyapp:1.0.0
```

```
The push refers to repository [localhost:5001/tinyapp]
```

```
...
```

```
b59ec4124c64: Layer already exists
```

```
e91e57f70610: Layer already exists
```

```
...
```

```
826b96dc18b7: Pushed
```

```
d122ce05d523: Layer already exists
```

```
1.0.0: digest:
```

```
↳ sha256:8615741e44becf529f9830f67909fedec585c424cc931db80f5df3e74a65d2ae
```

```
↳ size: 856
```

Only one layer was uploaded. The others were already in the registry from Chapter 6, because both images share the same base and libraries. That is layer sharing saving you time and space.

Ask the registry which tags it holds:

```
curl localhost:5001/v2/tinyapp/tags/list
```

```
{"name": "tinyapp", "tags": ["53ba61aa33f5...", "1.0.0", "10ff8b9d01b5..."]}
```

The long tags are the commit hashes from Chapter 6's pipeline. Yours will differ.

Step 7: Run It “Somewhere Else”

A real second machine would log in to a registry it can reach over the network and pull from it. You can get the same effect on your machine by deleting every local copy of the image first, so Docker has to fetch it:

```

docker rm -f tinyapp
docker rmi tinyapp:1.0.0 localhost:5001/tinyapp:1.0.0
docker run -d --name tinyapp -p 8080:8000 -e APP_VERSION=1.0.0
    ↪ localhost:5001/tinyapp:1.0.0

...
Untagged: tinyapp:1.0.0
Untagged: localhost:5001/tinyapp:1.0.0
Deleted: sha256:8615741e44becf529f9830f67909fedec585c424cc931db80f5df3e74a65d2ae
Unable to find image 'localhost:5001/tinyapp:1.0.0' locally
1.0.0: Pulling from tinyapp
826b96dc18b7: Download complete
Digest: sha256:8615741e44becf529f9830f67909fedec585c424cc931db80f5df3e74a65d2ae
Status: Downloaded newer image for localhost:5001/tinyapp:1.0.0
5b5efbbac133aa3edcade642bf004fc9edf08de1cdde20c7708813c16ad8433c

```

The digest after the pull is the same as the digest printed by the push: the bytes you run are exactly the bytes you built. After a few seconds:

```

docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
curl localhost:8080/health

NAMES      IMAGE                                STATUS
tinyapp    localhost:5001/tinyapp:1.0.0      Up 7 seconds (healthy)
{"status":"ok","version":"1.0.0"}

```

Leave it running if you're going straight on to Chapter 10, which replaces it.

When Something Goes Wrong

You see	It usually means
Bind for 0.0.0.0:8080 failed: port is already allocated	Another container already uses port 8080. Remove it, or pick another host port
curl fails to connect to port 8080	The container isn't running (check <code>docker ps -a</code> and <code>docker logs</code>), or <code>-p</code> is missing
(unhealthy) in <code>docker ps</code>	The app doesn't answer on port 8000; read the health log as in step 5
...tinyapp:9.9.9: not found when pulling	That tag was never pushed; list the tags with <code>curl</code>
pull access denied for tinyap, repository does not exist	A typo in the image name, so Docker looked for it on Docker Hub

Summary, Key Terms, and Review Questions

Summary

- Test first, then package. The image should contain only code that has passed its tests.
- A HEALTHCHECK lets Docker tell a running container from a working one.
- Tag images with a version, and with the registry's address so they can be pushed.
- Pushing uploads only missing layers; pulling by tag gives the same digest, so the same image.

Key Terms

Term	Meaning
Semantic versioning	Version numbers in the form MAJOR.MINOR.PATCH

Review Questions

1. Why is “the process is running” not enough to know the app works?
2. What do the four HEALTHCHECK options control?
3. Why did the push upload only one layer?
4. How can you be sure that the image you pulled is the one you built?

You understand this when you can ship a small web app, with a health check, through a registry.

Lab: Deploy on Every Push

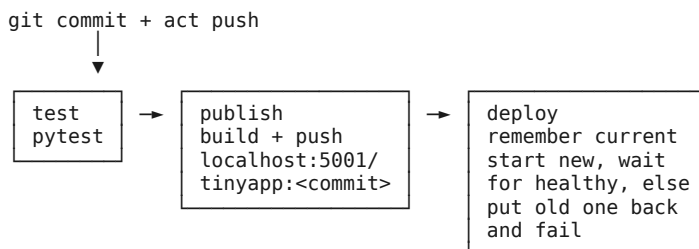
In this lab you build continuous deployment. Every push to `main` runs the tests, builds and publishes an image, and replaces the running app with it. Then you break a release on purpose and watch the pipeline notice and put the previous version back on its own. Finally, you roll back by hand and fix forward.

The problem. I want every change on `main` to go live on its own, safely.

The question. How do I build a pipeline that deploys every push, notices a bad release, and puts the good one back?

The Plan

“Production” in this lab is the container named `tinyapp` on port 8080 of your machine. The pipeline has three jobs:



The deploy job uses the `recreate` strategy from Chapter 4, with an **automatic rollback**: if the new container isn’t healthy in time, the old image goes back and the pipeline turns red.

Start in a fresh folder, `~/infra-labs/ch10`, with the app’s four files, Chapter 9’s `Dockerfile` (the one with the `HEALTHCHECK`), `.dockerignore`, and `.gitignore`, all from the book’s `labs/tinyapp/` folder, plus Chapter 6’s `.actrc` from `labs/ch06/`. Make it a repository with `git init -b main`. The registry on port 5001 must be running, as in Chapter 9.

Step 1: The Deploy Script

Deploying is several commands with decisions between them, so it lives in a script, `deploy.sh`, in the repository. The workflow calls it, and so can you.

```
#!/usr/bin/env bash
# Replace the running tinyapp container with a new image.
# If the new one is not healthy in time, put the old one back.
set -euo pipefail

NEW_IMAGE="$1"
NAME="tinyapp"

start() {
  docker rm -f "$NAME" >/dev/null 2>&1 || true
  docker run -d --name "$NAME" -p 8080:8000 \
    -e APP_VERSION="${1##*:}" "$1" >/dev/null
}

wait_until_healthy() {
  for _ in $(seq 1 30); do
    status="$(docker inspect --format '{{.State.Health.Status}}' "$NAME")"
    echo "  health: $status"
    case "$status" in
      healthy) return 0 ;;
      unhealthy) return 1 ;;
    esac
    sleep 2
  done
  return 1
}

OLD_IMAGE="$(docker inspect --format '{{.Config.Image}}' "$NAME" 2>/dev/null ||
  ↪ true)"
echo "Running now: ${OLD_IMAGE:-nothing}"
echo "Deploying:   $NEW_IMAGE"
start "$NEW_IMAGE"

if wait_until_healthy; then
  echo "Deploy succeeded: $NEW_IMAGE is healthy"
  exit 0
fi

echo "Deploy FAILED: $NEW_IMAGE is not healthy"
if [ -n "$OLD_IMAGE" ]; then
  echo "Rolling back to $OLD_IMAGE"
```

```

    start "$OLD_IMAGE"
    wait_until_healthy && echo "Rollback succeeded"
fi
exit 1

```

Reading it from the bottom half up:

- Before touching anything, it asks Docker which image the current `tinyapp` container runs, and remembers it as `OLD_IMAGE`. If there is no container yet, that's empty.
- `start` removes the old container and runs a new one. `${1##*:}` is the part of the image name after the last colon, the tag, which becomes `APP_VERSION`.
- `wait_until_healthy` asks for the health status every 2 seconds, for up to a minute, and stops as soon as it's healthy or unhealthy.
- On failure, it starts `OLD_IMAGE` again and exits with 1, which makes the pipeline step fail. A rollback that works still leaves the pipeline red, because the release didn't go out.

Make it executable with `chmod +x deploy.sh`.

Step 2: The Workflow

The workflow does everything Chapter 6's `ci.yml` did (apart from the secret check) and adds the deploy job. Save it as `.github/workflows/deploy.yml`:

```

name: Deploy

on:
  push:
    branches: [main]

concurrency: deploy

env:
  IMAGE: localhost:5001/tinyapp

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements-dev.txt
      - run: pytest -q

```

```

publish:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Build and push the image
      run: |
        TAG="${GITHUB_SHA: :7}"
        docker build -t "$IMAGE:$TAG" .
        docker push "$IMAGE:$TAG"

deploy:
  needs: publish
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Replace the running container
      run: ./deploy.sh "$IMAGE:${GITHUB_SHA: :7}"

```

GITHUB_SHA is the commit hash, and `${GITHUB_SHA: :7}` keeps its first seven characters, the short form git itself shows. concurrency: deploy tells GitHub never to run two of these workflows at once, so two quick pushes can't deploy over each other.

Step 3: The First Deploy

Remove any tinyapp container left over from Chapter 9, so the pipeline starts from nothing, then commit and push:

```

docker rm -f tinyapp
git add -A
git commit -m "Deploy on every push"
act push

```

The interesting lines of act's output:

```
[Deploy/test]    | 2 passed in 0.12s
[Deploy/test]    | 🟩 Job succeeded
[Deploy/publish] | 11e42a4: digest: sha256:03d58610... size: 856
[Deploy/publish] | 🟩 Job succeeded
[Deploy/deploy ] | Running now: nothing
[Deploy/deploy ] | Deploying:  localhost:5001/tinyapp:11e42a4
[Deploy/deploy ] | health: starting
[Deploy/deploy ] | health: starting
[Deploy/deploy ] | health: starting
[Deploy/deploy ] | health: healthy
[Deploy/deploy ] | Deploy succeeded: localhost:5001/tinyapp:11e42a4 is healthy
[Deploy/deploy ] | 🟩 Job succeeded
```

Production is live:

```
curl localhost:8080/health

{"status":"ok","version":"11e42a4"}
```

Step 4: A Good Change Goes Live

In `main.py`, make `/health` name the app:

```
    return {"status": "ok", "version": VERSION, "app": "tinyapp"}

git commit -am "Name the app in /health"
act push
```

In act's output:

```
[Deploy/deploy ] | Running now: localhost:5001/tinyapp:11e42a4
[Deploy/deploy ] | Deploying:  localhost:5001/tinyapp:6c357a4
...
[Deploy/deploy ] | Deploy succeeded: localhost:5001/tinyapp:6c357a4 is healthy
```

`curl localhost:8080/health` now returns `{"status":"ok","version":"6c357a4", "app":"tinyapp"}`. You changed code and committed; the machine did the rest.

Step 5: A Bad Deploy, Caught and Rolled Back

Now a change that passes every test but breaks production. Someone edits the Dockerfile's last line to serve on port 8001, forgetting that the health check (and the `-p` mapping) expect 8000:

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8001"]

git commit -am "Serve on port 8001"
act push
```

In act's output:

```
[Deploy/test] | 2 passed in 0.12s
[Deploy/test] 🟩 Job succeeded
[Deploy/publish] 🟩 Job succeeded
[Deploy/deploy ] | Running now: localhost:5001/tinyapp:6c357a4
[Deploy/deploy ] | Deploying: localhost:5001/tinyapp:17541b3
[Deploy/deploy ] | health: starting
...
[Deploy/deploy ] | health: unhealthy
[Deploy/deploy ] | Deploy FAILED: localhost:5001/tinyapp:17541b3 is not healthy
[Deploy/deploy ] | Rolling back to localhost:5001/tinyapp:6c357a4
[Deploy/deploy ] | health: starting
[Deploy/deploy ] | health: starting
[Deploy/deploy ] | health: starting
[Deploy/deploy ] | health: healthy
[Deploy/deploy ] | Rollback succeeded
[Deploy/deploy ] ❌ Failure - Main Replace the running container
↳ [23.091493666s]
[Deploy/deploy ] 🟩 Job failed
Error: Job 'deploy' failed
```

The tests passed because they never look at the Dockerfile. The image built and was published. Only the health check, in the real environment, caught the problem. Users saw a few seconds of downtime while the broken version was tried, and then the previous version was back:

```
docker ps --filter name=tinyapp --format 'table
↳ [{.Names}]\t[{.Image}]\t[{.Status}]'
curl localhost:8080/health
```

NAMES	IMAGE	STATUS
tinyapp	localhost:5001/tinyapp:6c357a4	Up 6 seconds (healthy)
{ "status": "ok", "version": "6c357a4", "app": "tinyapp" }		



A blue-green or rolling deployment would avoid even those few seconds: the new version starts next to the old one, and traffic moves only when it is healthy. The idea is the same; the platform does more of the work.

Step 6: Roll Back by Hand

Sometimes a release is healthy but wrong, and you decide to go back. Every version the pipeline ever built is still in the registry:

```
curl localhost:5001/v2/tinyapp/tags/list

{"name": "tinyapp", "tags": ["6c357a4", "53ba61aa33f5...", "11e42a4", "1.0.0",
↪ "10ff8b9d01b5...", "17541b3"]}]}
```

Rolling back is just deploying an older tag. The same script works from your terminal:

```
./deploy.sh localhost:5001/tinyapp:11e42a4

Running now: localhost:5001/tinyapp:6c357a4
Deploying:   localhost:5001/tinyapp:11e42a4
  health: starting
  health: starting
  health: starting
  health: healthy
Deploy succeeded: localhost:5001/tinyapp:11e42a4 is healthy
```

Step 7: Fix Forward

The real fix belongs in git, so that `main` describes what should run. `git revert` makes a new commit that undoes the bad one, and the pipeline deploys it like any other change:

```
git revert --no-edit HEAD
act push
```

The deploy job replaces the hand-picked image with one built from the revert commit, and ends with `Deploy succeeded`. The history in `git log --oneline` now tells the whole story: the good change, the bad one, and its revert.

How act Reaches Your Docker

The deploy job runs inside act's runner, yet `tinyapp` appears on your own port 8080. Through the Docker socket (Chapter 6), the job's `docker run` asks *your* Docker daemon to start the container, beside the runner rather than inside it.



Access to the Docker socket is access to the whole machine: anything that can talk to it can start containers with any files mounted. That is fine on your laptop. On shared CI, deploy jobs instead call the target platform with a narrowly scoped credential, for example a cloud deploy command, or SSH to a server that runs `docker compose up -d`.

Summary, Key Terms, and Review Questions

Summary

- Continuous deployment chains test, publish, and deploy; every passing push to `main` goes live.
- The deploy script remembers the running image, starts the new one, and waits for it to become healthy.
- Tests can pass while a release is broken. A health check in the real environment is the last line of defence, and it triggers the automatic rollback.
- Kept images make a manual rollback one command; `git revert` fixes forward and keeps `main` truthful.

Key Terms

Term	Meaning
Automatic rollback	Restoring the previous version when the new one fails its health check

Review Questions

1. What does `deploy.sh` remember before it changes anything, and why?
2. Why did the bad release pass the test and publish jobs?
3. The rollback in step 5 succeeded. Why is the pipeline still red?
4. What is the difference between rolling back (step 6) and fixing forward (step 7)?
5. How does a job inside act start a container on your machine, and why is that risky?

You understand this when you can push a change and predict, before act finishes, which image will be running on port 8080 and what the pipeline's final colour will be.

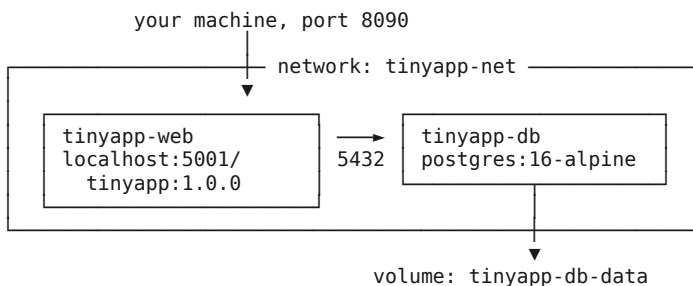
Lab: Rebuild Everything from Code

In this lab Terraform builds a complete small system: tinyapp, a PostgreSQL database, a private network between them, and a volume for the data. Then you change it, “lose the server”, get it all back with one command, and finally destroy and recreate everything, learning on the way what code can rebuild and what it can’t.

The problem. My server died. Can I get it all back in five minutes?

The question. If everything is described in code, can I delete it all and rebuild it with one command?

What You’ll Build



Six resources: two images, two containers, one network, one volume. The app image is the 1.0.0 you pushed in Chapter 9, so the registry must be running. If `curl localhost:5001/v2/tinyapp/tags/list` doesn’t list 1.0.0, build and push it from the book’s labs/tinyapp/ folder with `docker build -t localhost:5001/tinyapp:1.0.0 .` and `docker push localhost:5001/tinyapp:1.0.0`. This system uses port 8090 and its own names, so it doesn’t disturb Chapter 10’s tinyapp on port 8080.

Step 1: Describe the System

Create `~/infra-labs/ch11/main.tf`. It starts like Chapter 8's file, then declares two inputs:

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 4.6"
    }
  }
}

provider "docker" {}

variable "app_version" {
  type     = string
  default = "1.0.0"
}

variable "db_password" {
  type       = string
  sensitive = true
}
```

A **variable** is an input to a configuration. `app_version` has a default. `db_password` has none, so Terraform must be given it, and `sensitive = true` stops Terraform printing it. Terraform reads any environment variable called `TF_VAR_<name>`, so the password never has to be written in a file.

Next, the network, the volume, and the two images:

```
resource "docker_network" "lab" {
  name = "tinyapp-net"
}

resource "docker_volume" "db_data" {
  name = "tinyapp-db-data"
}

resource "docker_image" "postgres" {
  name          = "postgres:16-alpine"
  keep_locally = true
}
```

```
resource "docker_image" "app" {
  name      = "localhost:5001/tinyapp:1.0.0"
  keep_locally = true
}
```

The database container joins the network, keeps its files in the volume, and has a health check. `wait = true` makes Terraform wait until the container is healthy before moving on:

```
resource "docker_container" "db" {
  name      = "tinyapp-db"
  image     = docker_image.postgres.image_id
  env       = [
    "POSTGRES_USER=tinyapp",
    "POSTGRES_PASSWORD=${var.db_password}",
    "POSTGRES_DB=tinyapp",
  ]

  networks_advanced {
    name = docker_network.lab.name
  }

  volumes {
    volume_name      = docker_volume.db_data.name
    container_path   = "/var/lib/postgresql/data"
  }

  healthcheck {
    test      = ["CMD", "pg_isready", "-U", "tinyapp"]
    interval  = "2s"
    retries   = 10
  }
  wait = true
}
```

Finally, the app and an output:

```
resource "docker_container" "app" {
  name      = "tinyapp-web"
  image     = docker_image.app.image_id
  env       = [
    "APP_VERSION=${var.app_version}",
    "DATABASE_URL=postgresql://tinyapp:${var.db_password}@${docker_container.db.
      ↪ name}:5432/tinyapp",
  ]

  networks_advanced {
```

```

    name = docker_network.lab.name
  }

  ports {
    internal = 8000
    external = 8090
  }
  wait = true
}

output "url" {
  value = "http://localhost:8090"
}

```

`${docker_container.db.name}` is a reference inside the database address. It gives the app the database's name on the network, and it tells Terraform to create the database first. `wait = true` here waits for the image's own HEALTHCHECK from Chapter 9. An **output** is a value Terraform prints after apply and shows again with `terraform output`.

Step 2: Build It

```

cd ~/infra-labs/ch11
export TF_VAR_db_password=change-me-locally
terraform init
terraform plan

```

```

...
# docker_container.app will be created
# docker_container.db will be created
# docker_image.app will be created
# docker_image.postgres will be created
# docker_network.lab will be created
# docker_volume.db_data will be created
...
Plan: 6 to add, 0 to change, 0 to destroy.

```

```

Changes to Outputs:
+ url = "http://localhost:8090"

```

(These lines are picked out of a longer plan.) Apply it, answering yes. Putting `time` in front measures how long it takes:

```
time terraform apply

...
docker_network.lab: Creation complete after 2s [id=309f6b4865e3...]
docker_container.db: Creating...
docker_container.db: Creation complete after 4s [id=0506b66e5fb6...]
docker_container.app: Creating...
docker_container.app: Creation complete after 6s [id=19b358f8dfa7...]

Apply complete! Resources: 6 added, 0 changed, 0 destroyed.

Outputs:

url = "http://localhost:8090"
```

It took about 12 seconds; `time` adds its own lines, which differ between shells. Notice the order: Terraform created the network, volume, and images together, then the database, and only when the database was healthy, the app. Try it:

```
curl localhost:8090/health
curl localhost:8090/visits
curl localhost:8090/visits

{"status":"ok","version":"1.0.0"}
{"visits":1}
{"visits":2}
```

The count is now stored in PostgreSQL.

Step 3: Change Something

In `main.tf`, change the default of `app_version` from `"1.0.0"` to `"1.1.0"`, and plan:

```
# docker_container.app must be replaced
...
    ~ env                                = (sensitive value) # forces
      ↪ replacement
...
Plan: 1 to add, 0 to change, 1 to destroy.
```

Only the app container changes. Its environment is shown as (sensitive value) because it contains the password. Apply it, and Terraform destroys and re-creates just that container.

```
curl localhost:8090/health
curl localhost:8090/visits

{"status":"ok","version":"1.1.0"}
{"visits":3}
```

A new app container, and the count carried on, because it lives in the database, not in the app.

Step 4: The Server Dies

Simulate losing the machine's containers and network all at once:

```
docker rm -f tinyapp-web tinyapp-db
docker network rm tinyapp-net
```

`curl localhost:8090/health` now fails with a connection error: nothing is listening any more.

Ask Terraform what it thinks:

```
terraform plan

Note: Objects have changed outside of Terraform
...
# docker_container.db has been deleted
...
# docker_network.lab has been deleted
...
# docker_container.app will be created
# docker_container.db will be created
# docker_network.lab will be created
...
Plan: 3 to add, 0 to change, 0 to destroy.
```

Terraform compared its state with reality, found what was missing, and plans to put back exactly that. Apply it:

```
time terraform apply
```

```
...
```

```
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

About 14 seconds later:

```
curl localhost:8090/visits
```

```
{"visits":4}
```

Everything is back, and so is the data, because the volume was never deleted. That is the answer to the chapter's question: yes, in seconds, provided the data survives.

Step 5: Destroy and Recreate from Nothing

Now remove everything Terraform manages, volume included, answering yes:

```
terraform destroy
```

```
Plan: 0 to add, 0 to change, 6 to destroy.
```

```
...
```

```
docker_volume.db_data: Destruction complete after 2s
```

```
docker_network.lab: Destruction complete after 2s
```

```
Destroy complete! Resources: 6 destroyed.
```

Build it again from nothing:

```
terraform apply
```

```
curl localhost:8090/visits
```

```
...
```

```
Apply complete! Resources: 6 added, 0 changed, 0 destroyed.
```

```
...
```

```
{"visits":1}
```

The whole system is back in about 12 seconds, but the count starts again at 1. The code described an empty database, and that is what you got.



Infrastructure as code rebuilds infrastructure, not data. Data needs backups, stored somewhere else, and restores that you have actually tested. In a real cloud, managed databases take the backups for you; your job is to switch them on and try a restore.

Step 6: Protect the Data from Yourself

Terraform can refuse to destroy a resource. Add a `lifecycle` block to the volume:

```
resource "docker_volume" "db_data" {
  name = "tinyapp-db-data"

  lifecycle {
    prevent_destroy = true
  }
}
```

and try `terraform destroy` again:

```
Error: Instance cannot be destroyed
...
Resource docker_volume.db_data has lifecycle.prevent_destroy set, but the
plan calls for this resource to be destroyed.
...
```

Nothing was destroyed. Use this for anything whose loss would hurt: databases, buckets, volumes.

Step 7: See Where the Password Went

The password never appeared in `main.tf` or on screen. But look in the state file:

```
grep -c change-me-locally terraform.tfstate

2
```

It is stored there twice, in plain text. This is why state must never go into git, and why teams keep it in encrypted, access-controlled remote state.

To finish, remove the `lifecycle` block, then run `terraform destroy` and answer yes.

Summary, Key Terms, and Review Questions

Summary

- One Terraform configuration can describe a whole system: images, containers, a network, and a volume, with references deciding the order.
- Variables take inputs such as passwords from `TF_VAR_` environment variables; outputs print useful values.
- A change replaces only what it must. Data kept in a volume or database survives app replacements.
- When things vanish, `plan` detects it and `apply` rebuilds exactly what is missing, in seconds.
- Code rebuilds infrastructure, not data. Back data up, and use `prevent_destroy` for anything precious.
- State stores secrets in plain text; keep it private.

Key Terms

Term	Meaning
Variable (Terraform)	An input to a Terraform configuration
Output (Terraform)	A value Terraform prints after applying

Review Questions

1. How does Terraform know to create the database before the app?
2. After the “server died”, how did Terraform decide what to create, and why was the data still there?
3. What does `prevent_destroy` protect against, and what doesn’t it protect against?
4. Where did the database password end up, even though it was never in a file?

You understand this when you can delete every container and network of a system without worrying, because you know exactly what one `terraform apply` will bring back, and what it won’t.

PART IV

What Next

Security, Speed, Failure, and Cost

Every book in this series ends by asking four questions about its topic. For infrastructure, the answers pull together everything you’ve built: secrets and access, slow builds and cold starts, health checks and restarts, and the bill.

The problem. My app is live, and now I worry about what will leak, slow down, break, or cost too much.

The question. For the infrastructure around my app, what goes wrong in security, speed, failure, and cost, and what limits the damage?

Security: What Can Go Wrong, and What Limits the Damage

Leaked secrets. Passwords and keys escape through git commits, image layers, CI logs, and Terraform state. Keep them in a secret store (the CI system’s secrets, the cloud’s secret manager), pass them at run time, keep `.env`, `.secrets`, and `*.tfstate` out of git, and prefer roles, whose credentials expire by themselves. If a secret leaks, change it at once: deleting the commit doesn’t help, because copies already exist.

Too much access. One key that can do everything turns a small leak into a disaster. Apply least privilege as in Chapter 7: one identity per app and per environment, each with only the permissions it needs, and MFA on every human login.

Vulnerable images. Your image contains hundreds of packages you didn’t write, and security flaws are found in them every week. **Image scanning** compares every package in an image with public lists of known vulnerabilities. Trivy is a free scanner that runs in a container:

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  -v trivy-cache:/root/.cache aquasec/trivy image --quiet \
  --severity HIGH,CRITICAL --scanners vuln localhost:5001/tinyapp:1.0.0 | grep
  ↪ Total
```

```
Total: 44 (HIGH: 44, CRITICAL: 0)
```

The first run downloads Trivy’s vulnerability database into the `trivy-cache` volume (delete it with `docker volume rm trivy-cache` when you’re done). The same scan of the full `python:3.12` image found `Total: 343 (HIGH: 324, CRITICAL: 19)`, which is one more reason to start slim. Adding `--ignore-unfixed` shows only problems that already have a fix; for `tinyapp` on the day of writing, that was zero. The numbers change daily, so scan in the pipeline on every build, and rebuild images regularly even when your code hasn’t changed, to pick up fixed base images.

Open doors. Databases and admin ports open to the whole internet are found by scanners within minutes. Keep databases in private subnets, allow only the traffic each part needs, and put a load balancer in front of the app.

The supply chain. Every base image, library, and CI action is code from someone else running with your permissions. Pin versions (`python:3.12-slim`, `fastapi==0.141.1`, `kreuzwerker/docker ~> 4.6`), commit lock files, prefer official and widely used actions, and remember that a job with the Docker socket controls the machine.

Speed: What Makes This Slow

Slow builds. The biggest win is layer order: dependencies first, code last, so that the slow install step is `CACHED` on most builds (Chapter 5). Keep the build context small with `.dockerignore`. In CI, where each runner starts empty, use the cache options your CI offers: `actions/setup-python` can cache pip downloads, and Docker builds can reuse layers stored in the registry.

Slow pipelines. Run independent jobs in parallel, run the fastest checks first so failures show up early, and keep the test suite quick. A pipeline that takes an hour gets ignored; one that takes five minutes gets trusted.

Slow deploys. Every server pulls the image before it can start it. A 205 MB image arrives much sooner than a 1.62 GB one, and layers the server already has are skipped.

Cold starts. A new copy of an app can’t answer until it has started. On a laptop, `tinyapp` answered about 0.4 seconds after `docker run`, but serverless platforms and freshly created instances add their own start-up time, from a fraction of a second to many seconds for large images. Keep images small, do less work at start-up, and on serverless platforms keep a minimum number of copies warm if the delay hurts.

Distance. Every request crosses the network. Pick a region close to your users, and keep the app and its database in the same region.

Failure: How It Breaks, and How to Diagnose It

Crashes. A process can die: a bug, running out of memory, a restart of the machine. A restart policy tells Docker what to do then. Start a container with one:

```
docker run -d --name crashy --restart unless-stopped localhost:5001/tinyapp:1.0.0
```

Give the app a few seconds to start, then stop its process from inside, as if it had crashed. Signal 15 asks a process to stop:

```
docker exec crashy python -c "import os; os.kill(1, 15)"
```

Wait a few more seconds, then ask Docker what happened, and clean up:

```
docker inspect --format '{{.RestartCount}} restart(s), running:
↳ {{.State.Running}}' crashy
docker rm -f crashy

1 restart(s), running: true
crashy
```

`unless-stopped` restarts the container whenever it exits, until you stop it yourself. Use it (or `always`) for anything that should keep running.

Running but broken. A restart policy only notices a process that has exited. A stuck or misconfigured app needs a health check, and something that acts on it. Plain Docker only reports unhealthy; container platforms stop sending it traffic and replace it.

Bad releases. The pipeline is your safety net: tests before build, health-gated deploys, automatic rollback, and every old image kept in the registry (Chapter 10).

Lost machines and zones. Infrastructure as code rebuilds machines in minutes (Chapter 11); backups bring back data; copies in two availability zones keep you running while that happens.

When something is wrong, find the failing layer before changing anything:

Symptom	First thing to run
The app doesn't answer	<code>docker ps -a</code> (is it running?), then <code>docker logs <name></code>
It runs but misbehaves	<code>docker inspect --format '{{json .State.Health}}' <name></code>
"Connection refused"	Check the <code>-p</code> mapping and that the app listens on <code>0.0.0.0</code>
A pipeline fails	Read the first red step's log; rerun it with <code>act -v</code> for detail
Something changed by itself	<code>terraform plan</code> to find drift

Cost: What Does This Cost Me?

Forgotten resources. The most common surprise bill is something nobody uses any more: a test VM, a disk left behind by a deleted VM, an old database snapshot, a load balancer for a finished experiment. Create experiments with Terraform and remove them with `terraform destroy`; label everything so the bill shows who owns it.

Always-on services. VMs, managed databases, and load balancers charge every hour whether or not traffic arrives. Turn development environments off at night, and choose serverless or scale-to-zero services for spiky or tiny workloads (Chapter 3's lab showed where the lines cross).

Growth you don't see. Every push in Chapter 10 added an image to the registry, and every log line is stored somewhere. Set retention rules: keep, say, the last 20 images and 30 days of logs.

Data leaving the cloud. Egress is charged per gigabyte. Serving large files to users, or copying data between regions or providers, can cost more than the servers.

Oversized machines. Start small, measure, then grow. A small app on an instance four times too big pays four times too much, forever.

And before any of this: set a budget alert (Chapter 7), and read it when it arrives.

Summary, Key Terms, and Review Questions

Summary

- Security: keep secrets out of git, images, and logs; give least privilege; scan images and rebuild them regularly; close unneeded ports; pin what you depend on.
- Speed: order layers for caching, keep images and contexts small, parallelise pipelines, and plan for cold starts and distance.

- Failure: restart policies handle crashes, health checks handle stuck apps, pipelines and rollback handle bad releases, and IaC plus backups handle lost machines. Diagnose layer by layer.
- Cost: delete what you don't use, switch off what you don't need, set retention and budgets, and watch egress.

Key Terms

Term	Meaning
Image scanning	Checking an image's packages against lists of known vulnerabilities

Review Questions

1. Name four places a secret can leak from in the setup you built, and how each is prevented.
2. Why can an image with no changes to your code still need rebuilding?
3. What makes a Docker build slow, and what is the single biggest fix?
4. What is the difference between what a restart policy catches and what a health check catches?
5. List three ways a cloud bill grows without anyone noticing.

You understand this when you can look at any part of your setup, from the Dockerfile to the Terraform state, and say what could leak, slow down, break, or cost money there, and what you would do about it.

Where to Go from Here

You can now package an app so it runs the same everywhere, ship it through a pipeline that tests, publishes, deploys, and rolls back on its own, and describe the infrastructure in code that rebuilds it in seconds. You have also seen what the cloud sells, how it charges, and how to keep access tight. This short chapter points to what comes next, and what can wait.

Kubernetes and What It's For

In this book, one machine ran your containers, and a shell script replaced them. That works for one app on one server. With twenty services on fifty machines, someone has to decide which container runs where, restart the ones that fail their health checks, replace them one at a time during a deploy, add copies when traffic grows, and route requests to whichever copies are healthy.

Kubernetes is the system that does this. Like Terraform, it is declarative: you write files saying “run three copies of `tinyapp:1.1.0`, each with 512 MB of memory, healthy when `/health` answers”, and Kubernetes keeps reality matching them, forever, across many machines. Every major cloud offers a managed version.

It is powerful, and it is a lot to learn and to run. You need it when you have many services, many machines, and a team to look after them. For one small app, a PaaS or a VM running Docker Compose is simpler, cheaper, and just as reliable. Everything you learned here, from images and health checks to declarative files and rolling deploys, is what Kubernetes is built on, so you'll find it much easier when the time comes.

Platform Engineering

As companies grow, every product team ends up needing the same things: a Dockerfile, a pipeline, a registry, Terraform for a database, alerts, and secrets. Platform engineering is the practice of building those once, well, as a product for the company's own developers.

The result is often called an internal developer platform: ready-made templates, pipelines, and infrastructure modules, so that a developer can go from a new service to production by following a paved path instead of becoming an infrastructure expert first.

The skills in this book are exactly the ones platform engineers use every day.

What to Learn Next, in This Order

1. *A real cloud, carefully.* Create a free-tier account on one provider, set a budget alert first, and deploy tinyapp to its container service (a PaaS). Then delete it.
2. *Terraform for that cloud.* Recreate the same deployment with Terraform, and keep the state in remote storage.
3. *Observability.* Observability is being able to see inside a running system from the outside, through logs, metrics, and alerts, and it's the missing piece in this book's setup. Learn to answer "is it working, and how do I know?" without logging into a machine.
4. *Networking.* DNS, HTTPS certificates, and load balancers are where many deploys get stuck. *Networking from Zero* covers them.
5. *Linux.* Servers are Linux machines. *Linux and Git from Zero* and *Operating Systems from Zero* go deeper into processes, files, and permissions.
6. *Kubernetes basics*, once you have several services to run.

What to Skip for Now

- Service meshes, which manage traffic between hundreds of services.
- Multi-cloud and multi-region designs. One cloud, one region, two zones will serve you for a long time.
- Writing your own CI runners, Kubernetes operators, or Terraform providers.
- Chasing every new tool. The ideas in this book (images, pipelines, declarative infrastructure, least privilege, health checks) outlast any single product.

Cleaning Up the Labs

Chapters 5, 7, 8, and 11 cleaned up after themselves. What is left is the app, the registry, the images you built, and Trivy’s cache. `-v` also deletes the registry’s storage:

```
docker rm -f -v tinyapp registry
docker image ls --format '{{.Repository}}:{{.Tag}}' | grep tinyapp | xargs docker
    ↪ rmi
docker volume rm trivy-cache

tinyapp
registry
Untagged: localhost:5001/tinyapp:58fc4c9
Deleted: sha256:32936c85afb2b26ec116bdb9c6eb2268cad469b4303b5a719cfa61d39444f7d
...
trivy-cache
```

The images you downloaded (Python, PostgreSQL, nginx, MinIO, Trivy, and act’s runner) stay; remove any of them with `docker rmi` if you need the disk space.

A Last Word

The problem at the start of this book was “it works on my machine but breaks on the server, and I don’t even own a server”. You now know why it breaks, how to make it not break, and how to rent, build, and rebuild the servers with a few commands. The next time something works on your machine, you’ll know how to make it work for everyone.

Docker Cheat Sheet

Images and Containers

Command	What it does
<code>docker build -t tinyapp:1.0.0 .</code>	Build an image from the Dockerfile here, and name it
<code>docker images, docker rmi IMAGE</code>	List images; delete one (or one of its names)
<code>docker tag IMG localhost:5001/IMG</code>	Add a registry address to an image's name
<code>docker push NAME, docker pull NAME</code>	Upload to, or download from, a registry
<code>docker run --rm IMAGE COMMAND</code>	Run a one-off container, removed when it exits
<code>docker run -d --name N -p 8080:8000 IMAGE</code>	Run in the background, forwarding host port 8080 to 8000
<code>... -e KEY=value, --env-file .env</code>	Pass configuration as environment variables
<code>... -v VOLUME:/path, -v "\$PWD":/path</code>	Attach a volume, or a folder from your machine
<code>... --restart unless-stopped</code>	Restart it whenever it exits, until you stop it
<code>docker ps, docker ps -a</code>	List running containers, or all of them
<code>docker logs N, docker exec N COMMAND</code>	See its output; run a command inside it
<code>docker inspect --format '{{.State.Health.Status}}' N</code>	Show one field, here the health status
<code>docker rm -f N</code>	Stop and remove a container
<code>docker volume create V, docker network create NET</code>	Create a volume or a network (rm deletes)

Compose

Command	What it does
<code>docker compose up -d</code>	Build if needed, then start everything in <code>compose.yaml</code>
<code>docker compose ps, logs SERVICE</code>	List the project's containers; show a service's output
<code>docker compose down</code>	Remove containers and networks; keep volumes (<code>-v</code> deletes them)

Dockerfile Instructions

Instruction	Purpose
<code>FROM image:tag</code>	The base image to start from
<code>WORKDIR /app, COPY src dest</code>	Set the working folder; copy files into the image
<code>RUN command</code>	Run a command while building
<code>ENV KEY=value</code>	Set an environment variable in the image (never a secret)
<code>USER name</code>	Run as this user instead of root
<code>EXPOSE 8000</code>	Document the port the app listens on
<code>HEALTHCHECK ... CMD command</code>	How Docker checks that the app works
<code>CMD ["program", "arg"]</code>	The command a container runs at start

CI and Terraform Cheat Sheet

GitHub Actions and act

Key or command	Meaning
<code>on:</code>	Events that start the workflow
<code>jobs.<id>.runs-on:</code>	The runner a job uses
<code>needs:</code>	Run this job only after the named jobs succeed
<code>uses: / with:</code>	Run a ready-made action / give it inputs
<code>run:</code>	Run shell commands (<code>run: </code> starts a block of several lines)
<code>env:</code>	Environment variables for the workflow, a job, or a step
<code>\${{ secrets.NAME }}</code>	A secret, masked as *** in logs
<code>\${{ github.sha }}</code> , <code>\$GITHUB_SHA</code>	The commit hash being built
<code>concurrency: NAME</code>	Never run two workflows in the same group at once
<code>act -l, act push</code>	List the jobs; run the workflows as if you had pushed (<code>-v</code> for detail)
<code>.actrc, .secrets</code>	act's default options; its secrets, one NAME=value per line (keep out of git)

Terraform Commands

Command	What it does
<code>terraform init</code>	Download providers and prepare the folder
<code>terraform validate</code>	Check the files for mistakes
<code>terraform plan</code>	Show what would change: + create, ~ update, - destroy, -/+ replace
<code>terraform apply</code>	Show the plan, ask for yes, and make the changes
<code>terraform destroy</code>	Delete everything the configuration manages, after asking
<code>terraform state list</code> , <code>terraform output</code>	List what's in the state; print the outputs again

Terraform Language

To do this	Write this
Choose a provider	<code>required_providers</code> in the <code>terraform</code> block, with source and version
Declare a resource	<code>resource "TYPE" "NAME" { ... }</code>
Refer to another resource	<code>docker_image.app.image_id</code> (type, name, attribute)
Use a variable	<code>var.app_version</code> , or <code>"APP_VERSION=\${var.app_version}"</code>
Set a variable from the shell	<code>export TF_VAR_db_password=...</code>
Hide a variable's value	<code>sensitive = true</code> in its variable block
Print a value after apply	an output block with a value
Refuse to destroy a resource	<code>lifecycle { prevent_destroy = true }</code>

Glossary

Term	Meaning
Access key	A key ID and secret a program uses to sign requests
Artifact	A pipeline's built, versioned output, such as an image
Automatic rollback	Restoring the previous version when a new one is unhealthy
Availability zone	An independent data centre within a region
Cloud	Computers and services rented over the internet by use
Configuration	Settings that differ between environments
Container	A program running with its own isolated files
Continuous delivery	Every passing change is ready to release
Continuous deployment	Every passing change is released automatically
Continuous integration (CI)	Building and testing every change automatically
Declarative	Describing the result, not the steps
Deployment strategy	How a new version replaces the old one
DevOps	Building and running software as one automated job
Digest	A sha256: fingerprint naming one exact image
Dockerfile	The recipe Docker follows to build an image
Drift	Real infrastructure differing from its code
Environment	One complete place an app runs, with its own settings
Environment variable	A named value handed to a process at start
Firewall	Rules deciding which traffic may pass
Health check	A quick automatic test that an app copy works
Identity and access management (IAM)	Decides who may do what in a cloud
Image	A read-only package of files and a start command
Image scanning	Checking an image for known vulnerabilities

Term	Meaning
Infrastructure	Everything code needs to run for others
Infrastructure as a Service (IaaS)	Renting virtual machines, disks, and networks
Infrastructure as code (IaC)	Infrastructure described in files a tool builds from
Layer	One build step's file changes in an image
Least privilege	Giving each identity only the permissions it needs
Load balancer	Spreads requests across healthy copies
Managed database	A database the provider runs and backs up
Object storage	Files stored by name in buckets, used over HTTP
Output (Terraform)	A value printed after apply
Pipeline	Automated stages every change passes through
Plan (Terraform)	A preview of the changes Terraform would make
Platform as a Service (PaaS)	A service that runs your code or container
Policy	A document of allowed actions on resources
Port	A number choosing which program receives traffic
Production	The environment real users use
Provider (Terraform)	A plugin that manages one system
Region	A geographic area of a provider's data centres
Registry	A server that stores and serves images
Resource (Terraform)	One thing a provider manages
Role	Permissions taken on with short-lived credentials
Rollback	Returning to the previous good version
Root user	A cloud account's all-powerful owner login
Secret	Private configuration, such as a password
Semantic versioning	Versions in the form MAJOR.MINOR.PATCH
Serverless	Code run per event, without managing servers
Shared responsibility model	Provider secures the platform; you secure your part
Staging	A production-like rehearsal environment
State (Terraform)	Terraform's record of what it built
Tag	A movable version label on an image
Variable (Terraform)	An input to a Terraform configuration
Virtual machine (VM)	Software imitating a whole computer
Virtual private cloud (VPC)	Your private network inside a cloud
Volume	Docker-managed storage that outlives containers
Workflow	One GitHub Actions pipeline, in a YAML file

Index

A

access key, 47
artifact, 24
automatic rollback, 68
availability zones, 16

C

cloud, 5
configuration, 4
container, 8
continuous delivery, 23
continuous deployment, 23
continuous integration, 23

D

declarative, 54
deployment strategy, 24
DevOps, 4
digest, 11
Dockerfile, 31
drift, 55

E

environment, 4
environment variable, 31

F

firewall, 18

H

health check, 24

I

identity and access management, 47
image, 9
image scanning, 86
infrastructure, 3
infrastructure as code, 53

L

layer, 10
least privilege, 47
load balancer, 18

M

managed database, 48

O

object storage, 17
output, 79

P

pipeline, 23
plan, 55
policy, 47
port, 18
production, 4
provider, 54

R

region, 16
registry, 11
resource, 54

role, 47
rollback, 25
root user, 46

S

secret, 33
semantic versioning, 63
serverless, 16
shared responsibility model, 15
staging, 4
state, 55

T

tag, 11

V

variable, 77
virtual machine, 10
virtual private cloud, 18
volume, 33

W

workflow, 39

Infrastructure Engineering from Zero

It works on your machine but breaks on the server, and you don't even own a server. This book starts from zero and shows how code gets from your laptop to machines that serve everyone, the same way every time. Every lab runs on your own computer, with no cloud account and no bill.

You'll learn how to:

- Package an app in a container that runs the same everywhere
- Build a pipeline that tests, builds, and deploys on every push
- Understand what you rent from a cloud provider, and what it costs
- Describe infrastructure in code, and rebuild it in minutes
- Roll back safely when a deploy goes wrong